

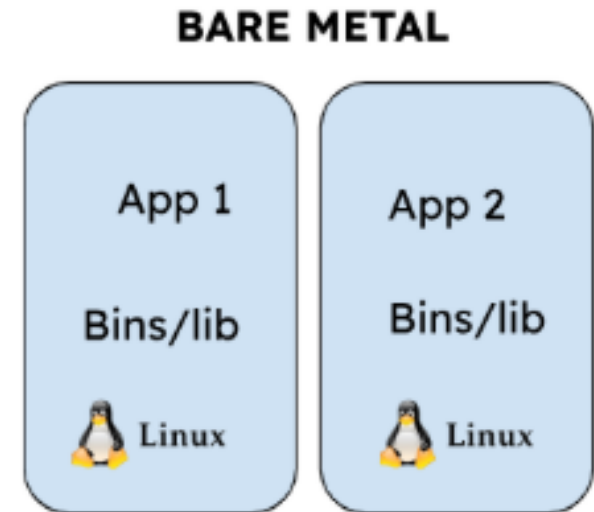
Docker 기본

inky4832@daum.net

1.1 서버 운영의 역사

물리 서버 시대 (Bare Metal)

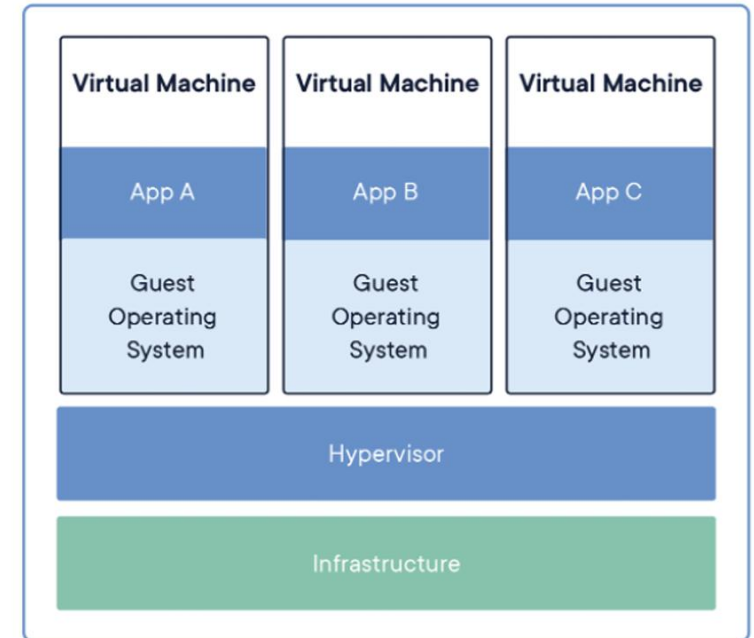
- 하나의 서버 = 하나의 운영체제 = 하나의 서비스
- 서버를 구축하면 하나의 프로그램만 실행
- 서버 자원이 남아도 활용 불가
- 문제점
 - 서버 비용 증가
 - 확장 어려움
 - 장애 발생 시 복구 느림



1.1 서버 운영의 역사

가상 서버 시대 (Virtual Machine)

- 하나의 물리서버 위에 여러 개의 가상 OS 실행
- 장점
 - 서버 자원 활용도 향상
 - 서비스 분리 가능
- 문제점
 - OS까지 포함(무겁다)
 - 부팅 느림
 - 디스크/메모리 사용량 큼

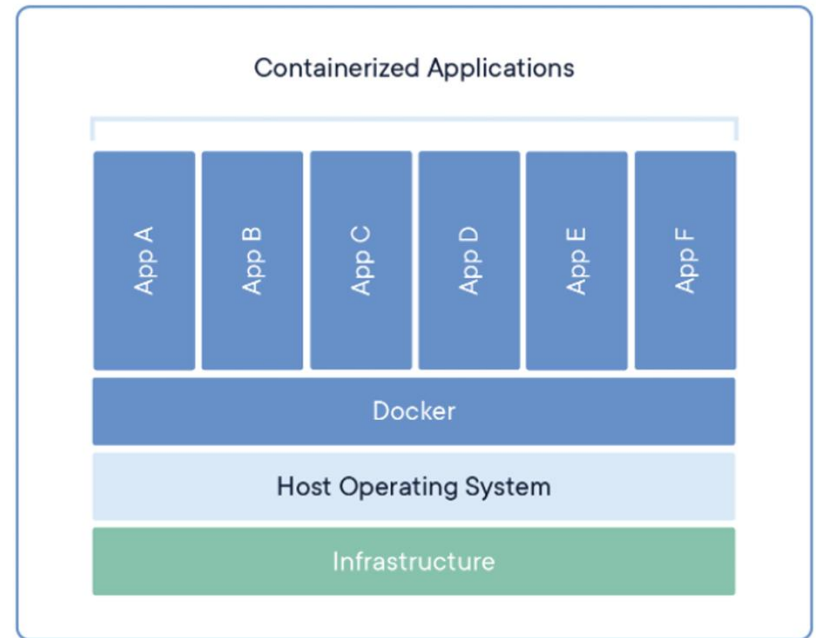


Virtual Machines

1.1 서버 운영의 역사

컨테이너 시대 (Container / Docker)

- OS 공유
- 어플리케이션과 실행 환경만 분리
- 장점
 - 가볍고 빠르다
 - 동일한 실행환경 보장
 - 매우 쉽게 재구축 및 배포 가능
- 특징
 - 표준화된 유닛(unit)
 - 매우 쉽게 move 가능
 - 구성 파일로 설명 및 공유 가능
 - self-containing
 - 즉, 외부 도움없이 독립적(자체적)으로 기능을 수행할 수 있음



Containers

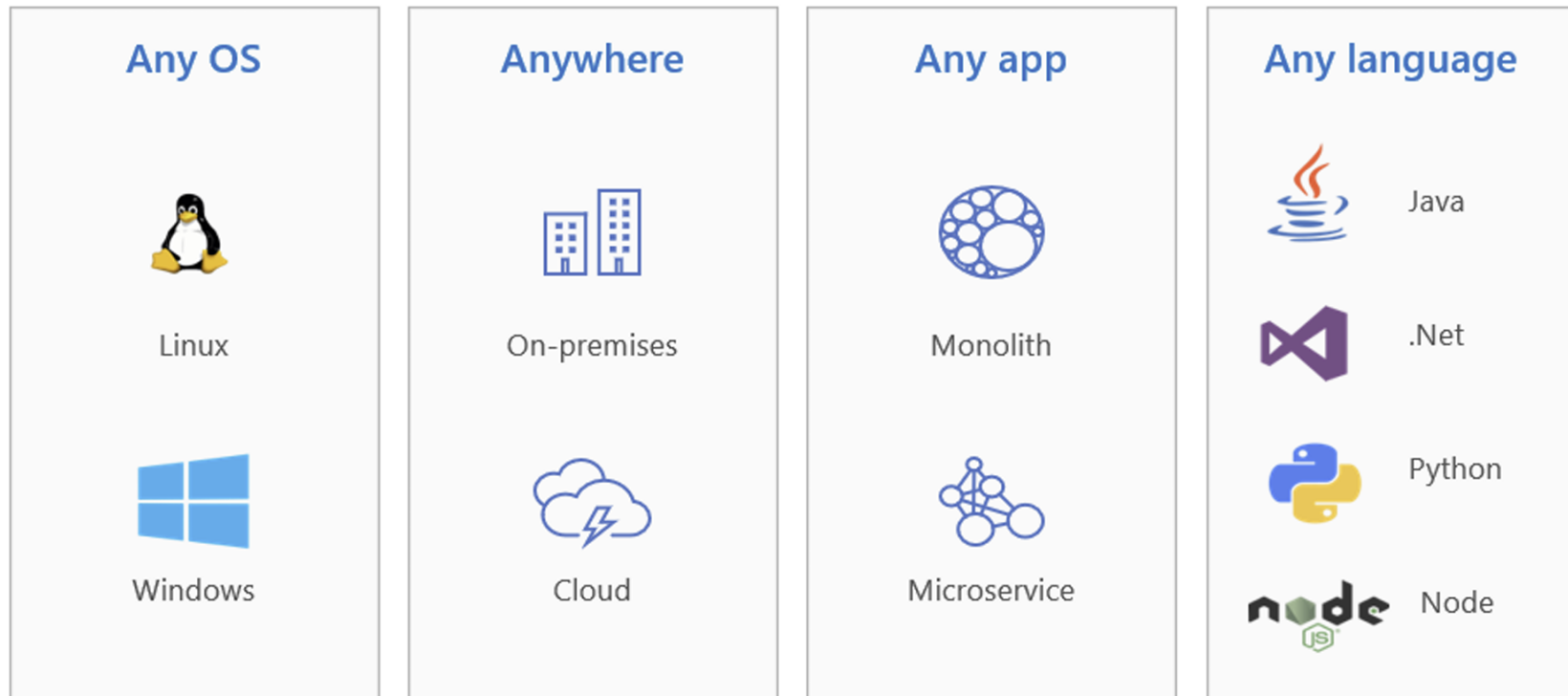
1.1 서버 운영의 역사

VM vs 컨테이너(Docker)

구분	VM	Docker
격리 단위	OS	프로세스
OS 포함	O	X
부팅시간	느림	빠름
용량	큼	작음
자원 사용	비효율	효율적
환경 재현	어려움	쉬움

1.1 서버 운영의 역사

컨테이너 장점



1.2 기존 방식의 문제점

실제 현업에서 가장 많이 나오는 문제

- "내 PC에서는 되는데 서버에서는 안됨"
- 상황 예시
 - 개발자 PC에서는 정상 동작
 - 테스트 서버에서는 에러 발생
 - 운영 서버에서는 실행조차 안됨
- 원인
 - OS 버전 차이
 - 라이브러리 버전 차이
 - 환경 변수 설정 차이
 - 설정 파일 누락

항목	문제
환경 설정	사람마다 다름
재현성	동일 환경 재현 불가
배포	매번 수동 작업
장애 대응	원인 파악 어려움

1.3 컨테이너 관련

OCI (Open Container Initiative)

- 컨테이너 표준을 만드는 조직
 - 역할
 - 이미지 형식 표준
 - 컨테이너 실행 방식 표준
- 즉 '컨테이너는 이렇게 만들어야 돼~" 규칙 정의

containerd

- 컨테이너 실행 엔진 (Runtime)
- 역할
 - 이미지 pull
 - 컨테이너 실행
 - LifeCycle 관리



1.3 컨테이너 관련

Docker

- 가장 유명한 컨테이너 플랫폼
- 내부적으로 containerd 사용
- 역할
 - 이미지 빌드 및 실행/관리

CRI-O

- K8s 전용 컨테이너 런타임
- 특징
 - 가볍고 단순함
 - K8s와 최적화

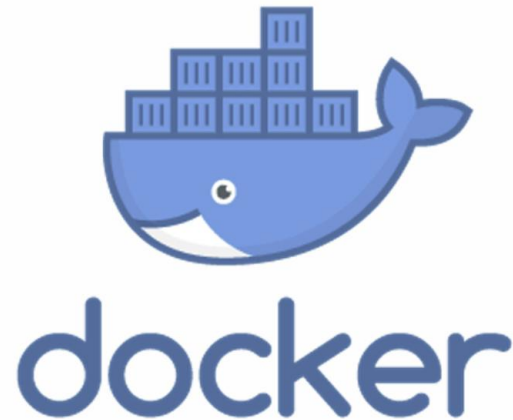
Podman

- Docker 대체 도구
- 특징
 - daemon 없음
 - docker CLI와 비슷

2.1 Docker 핵심 개념

Docker 개념

- 컨테이너를 만들고 관리하기 위한 오픈소스 소프트웨어.
- 컨테이너 기술을 활용하여 어플리케이션과 인프라를 분리. (독립적)
- Docker로 패키징 된 앱은 어디서든 실행 가능.
- 의존성 문제(환경 문제)가 사라져 개발자와 운영자가 같은 환경을 사용.



2.1 Docker 핵심 개념

Docker 핵심 아이디어

- "어플리케이션과 실행환경을 하나로 묶자"
ex. JDK + App 을 하나로.

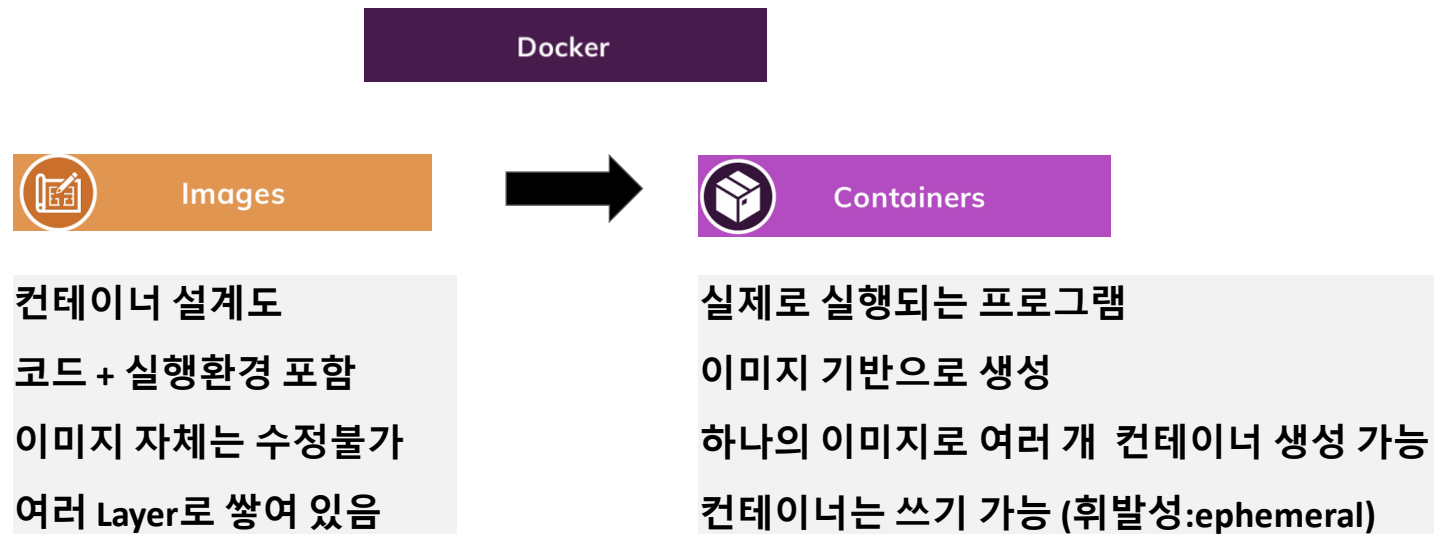
Docker 해결책

- 환경불일치 해결
 - 개발/테스트/운영 환경 동일
 - 어디서 실행해도 동일한 결과
 - ex. 내 PC = 테스트서버 = 운영서버
- 배포 표준화 및 자동화
 - 설치문서 대신에 Docker이미지로 대체
 - 실행방법이 항상 동일
- 빠른 배포 및 롤백
 - 컨테이너는 몇 초만에 실행
 - 필요시 이전 버전으로 즉시 복구 가능
- 마이크로서비스
 - 서비스 단위로 컨테이너 분리
 - 독립적 배포 및 장애 전파 최소화

2.1 Docker 핵심 개념

Docker 기본 흐름

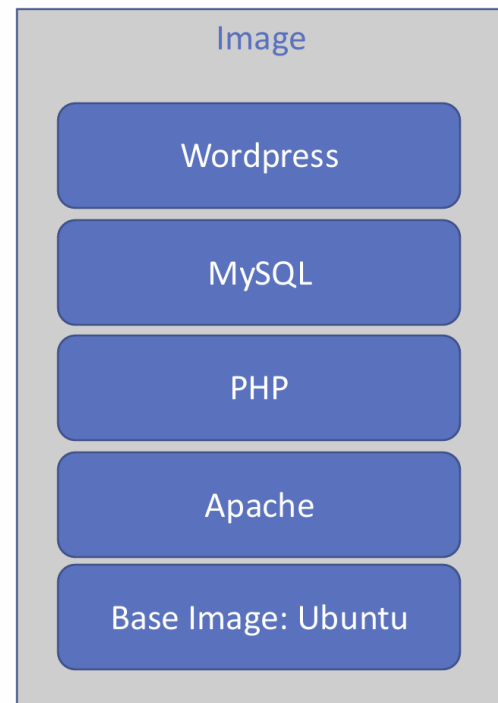
- Registry ----> Image ----> Container
- Docker는 이미지를 Registry로부터 다운 받아서 컨테이너로 실행.



2.1 Docker 핵심 개념

Docker Image

- 컨테이너를 만들기 위한 설계도 (템플릿) 역할.
- 구성요소
 - base 이미지(ex. ubuntu, mysql 등)
 - 라이브러리
 - 설정 파일
 - 실행 명령어 등
- 특징
 - 읽기 전용(Read-Only)
 - 실행되지 않는 정적(static)파일
 - 여러 컨테이너 생성 가능
 - 여러 레이어로 구성
 - 초기 layer를 base image라고 부름
 - base image에서 변경이 발생할 때마다 새로운 레이어가 추가로 생성됨.
 - 다른 이미지에 의존해서 만들어질 수 있음



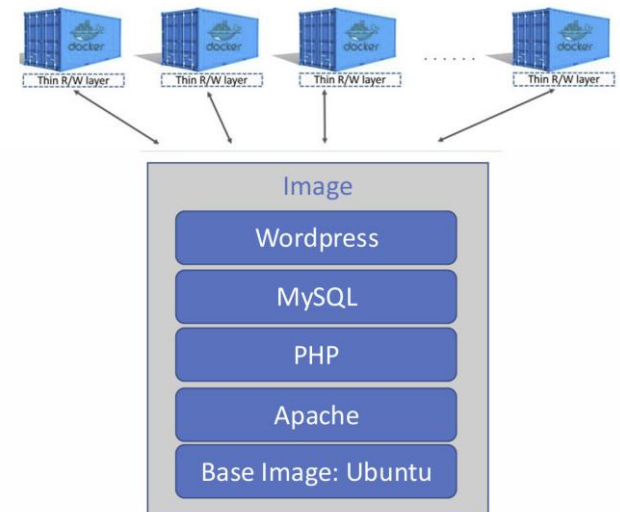
2.1 Docker 핵심 개념

Docker Container

- 개념
 - 이미지를 실행한 실제 실행 단위.
 - 격리된 환경에서 실행 중인 프로세스.

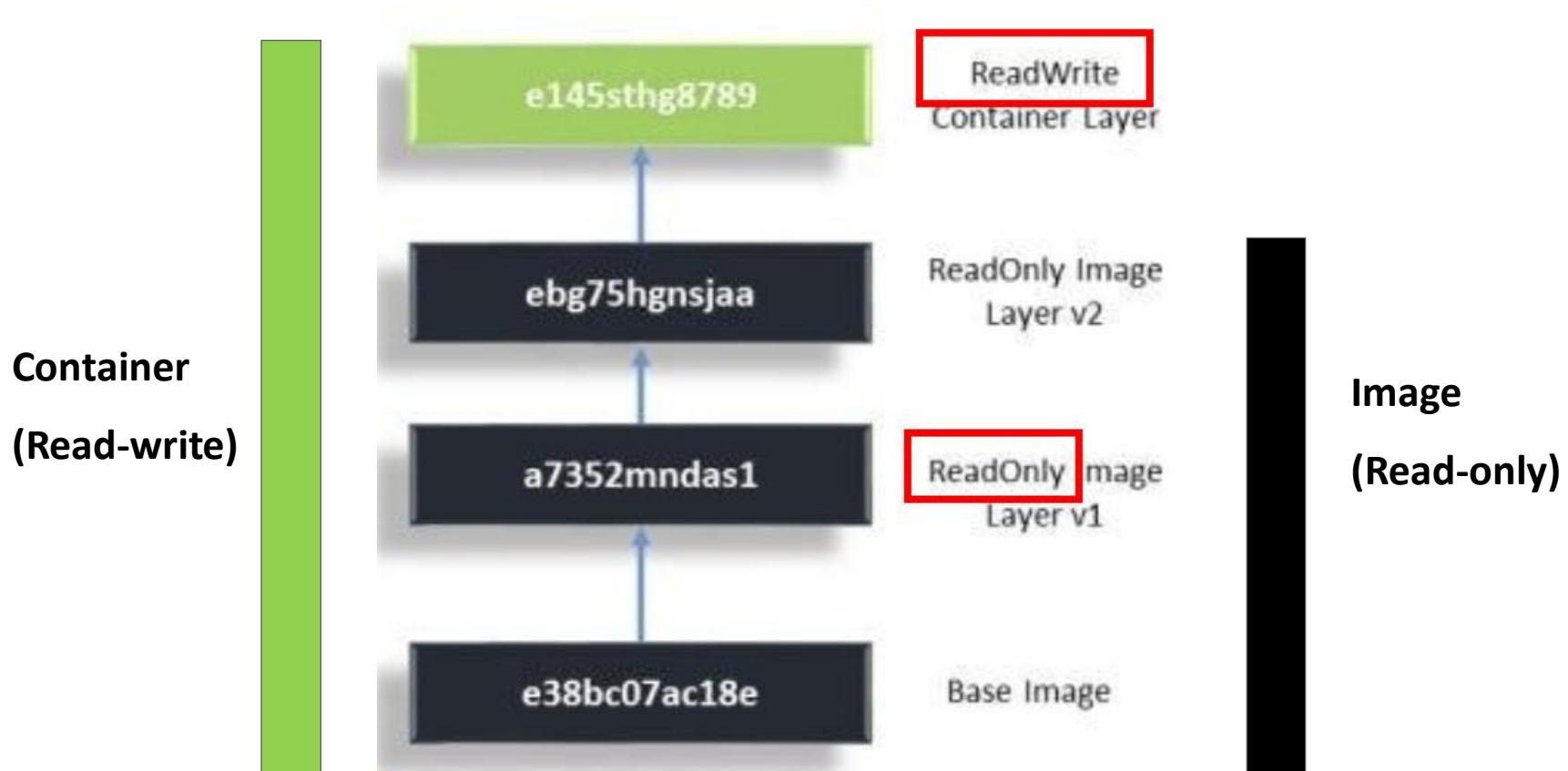
따라서 컨테이너에서 실행 중인 어플리케이션 직접 접근은 불가능
- 특징
 - 생성 / 시작 / 중지 / 삭제 가능
 - 상태를 가짐 (running / stopped)

구분	Image	Container
상태	정지	실행
수정	불가	가능 (휘발성)
개수	1개	여러 개
역할	설계도(템플릿)	실행 중인 프로세스



2.1 Docker 핵심 개념

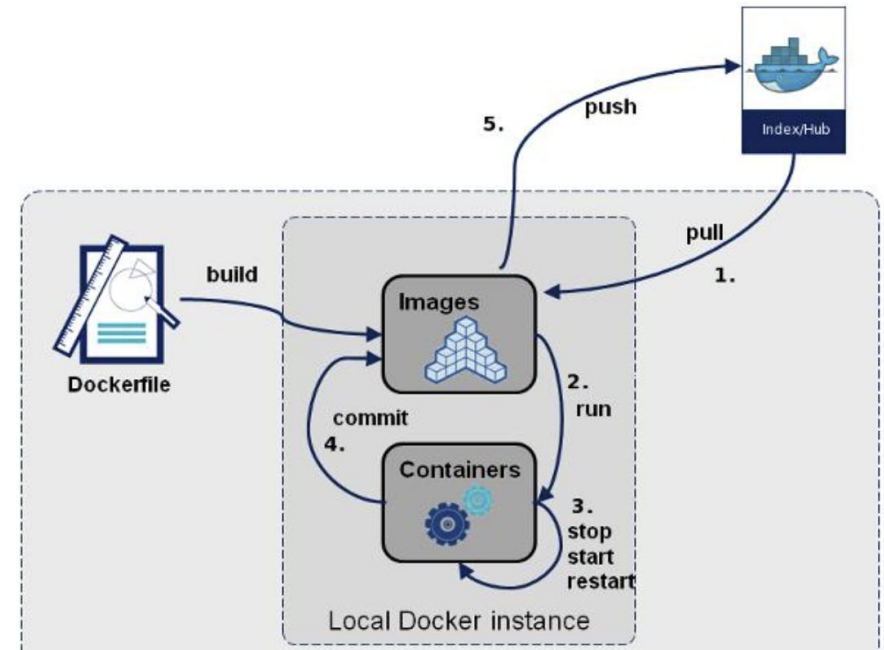
Image 와 Container



2.1 Docker 핵심 개념

Docker Registry

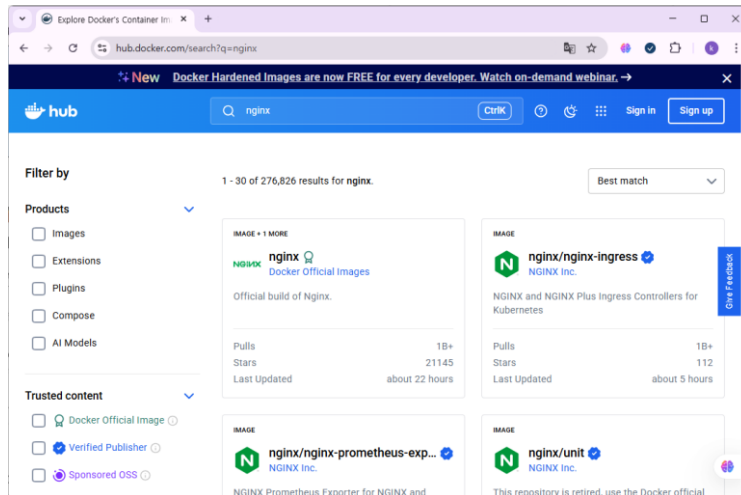
- 개념
 - Docker 이미지를 저장하는 서버
 - 이미지 저장소로서 공유 공간 기능
ex. Docker Hub (공식 지원), AWS ECR
- 주요 역할
 - 이미지 업로드 (push)
 - 이미지 다운로드 (pull)
 - 버전 관리
 - ex. `docker pull nginx`
`docker push my-image`



2.2 Docker Hub

Docker Hub

- Docker 공식 이미지 저장소
- 전 세계 개발자들이 사용
- 검증된 Official Image 제공
 - ex. nginx, mysql, redis, ubuntu 등
- **<https://hub.docker.com/>**



3.1 윈도우용 Docker 설치

Docker Desktop

- <https://www.docker.com/pricing/>

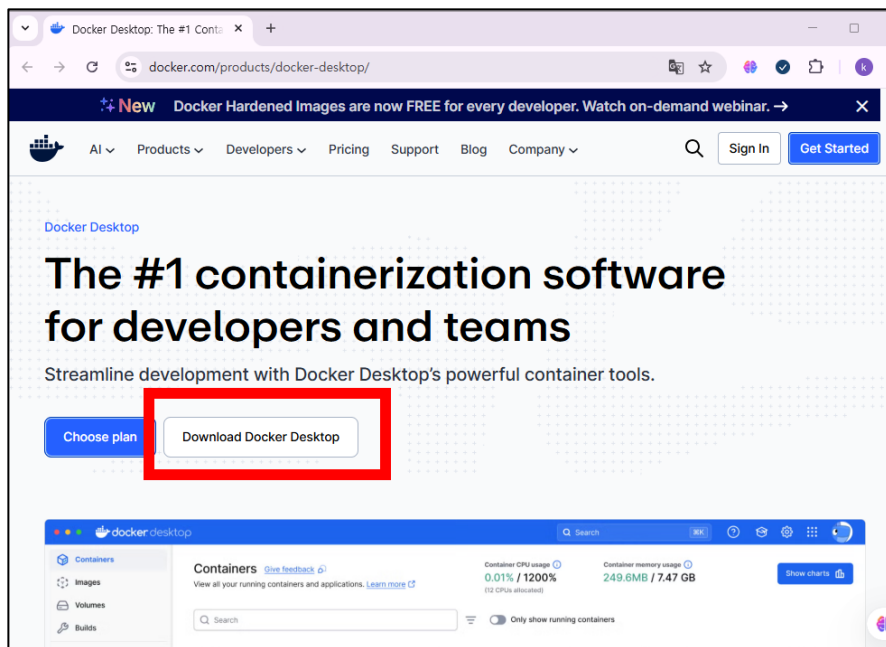
The screenshot displays the Docker pricing page with a navigation bar at the top containing links for AI, Products, Developers, Pricing, Support, Blog, and Company. A search icon, 'Sign In' button, and 'Get Started' button are also present. Below the navigation bar, there are tabs for 'Yearly' and 'Monthly' pricing. The main content area features four pricing plans:

Plan	Price	Target Audience	Key Features
Docker Personal	\$0	For individual developers who need the essential tools to build and deploy containers.	Docker Desktop, Docker Engine + Kubernetes, Docker Hub, Docker Scout, Docker Debug
Docker Pro	\$11 per user/month	For individual professionals who require more advanced features and additional resources.	Docker Build Cloud, Testcontainers Cloud, Synchronized File Shares, Visibility into Docker Scout health scores, 5 business day support response
Docker Team (Most Popular)	\$16 per user/month	For small teams that need collaborative tools to make working together more efficient.	Add users in bulk, Audit logs, Docker Hub role-based access control, 2 business day support response
Docker Business	\$24 per user/month	For enterprises desiring robust security, control, and compliance features.	Hardened Docker Desktop, Single Sign-On (SSO), SCIM user provisioning, Image and Registry Access Management, Desktop Insights Dashboard

3.1 윈도우용 Docker 설치

Docker Desktop 다운로드 및 설치

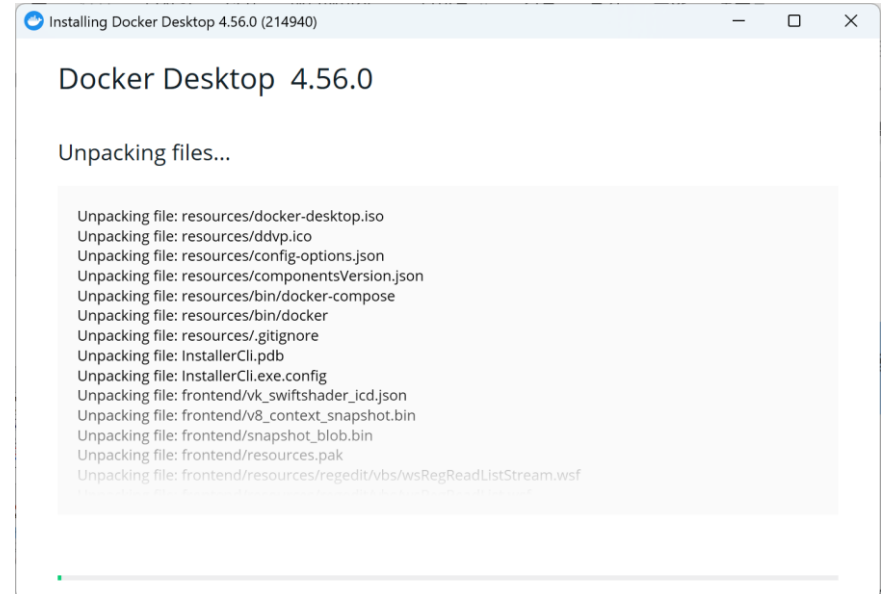
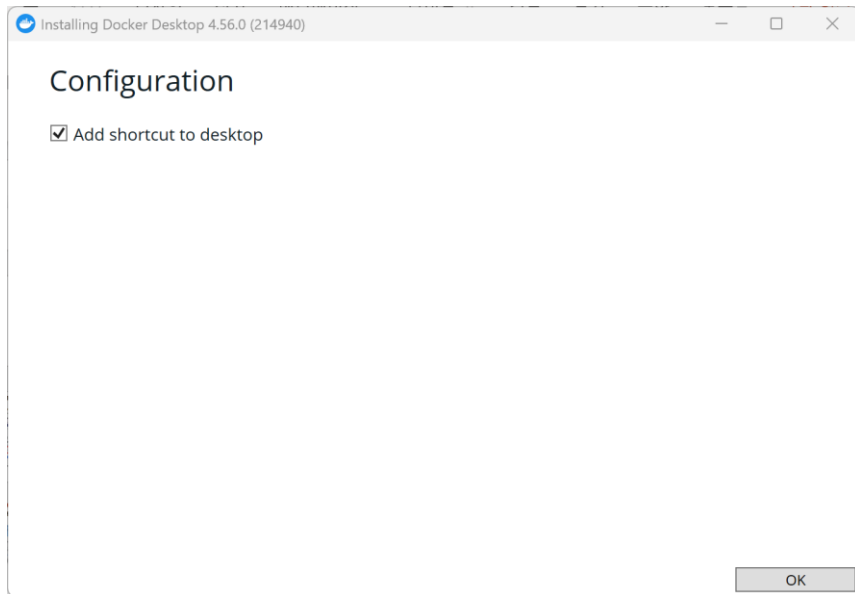
- <https://www.docker.com/products/docker-desktop/>



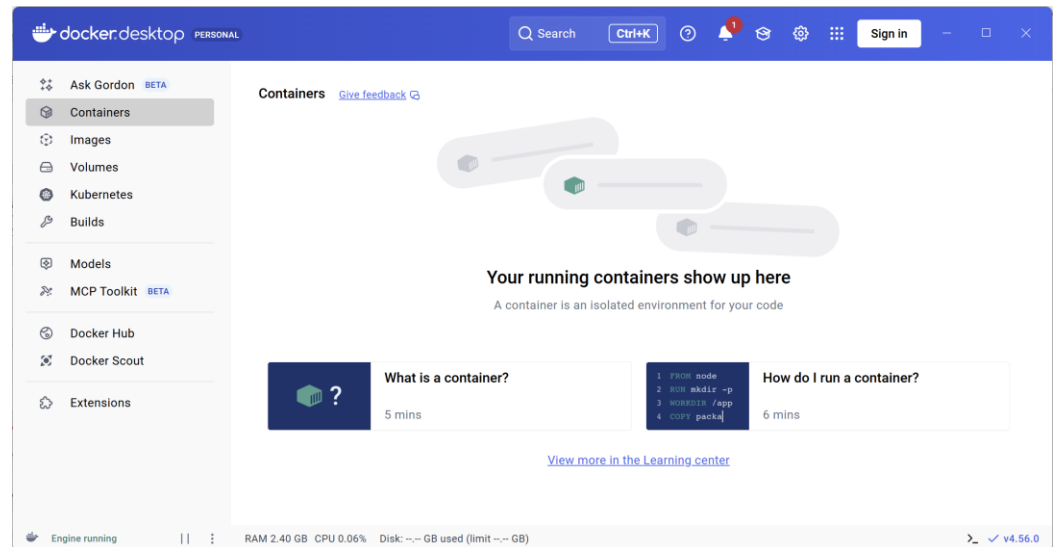
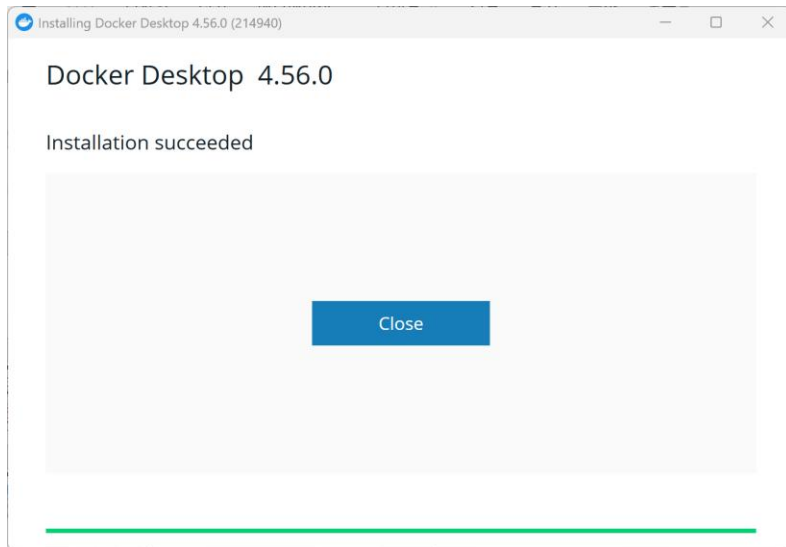
■ Docker 설치 문서

- <https://docs.docker.com/desktop/setup/install/windows-install/>
- <https://docs.docker.com/desktop/setup/install/mac-install/>

3.1 윈도우용 Docker 설치

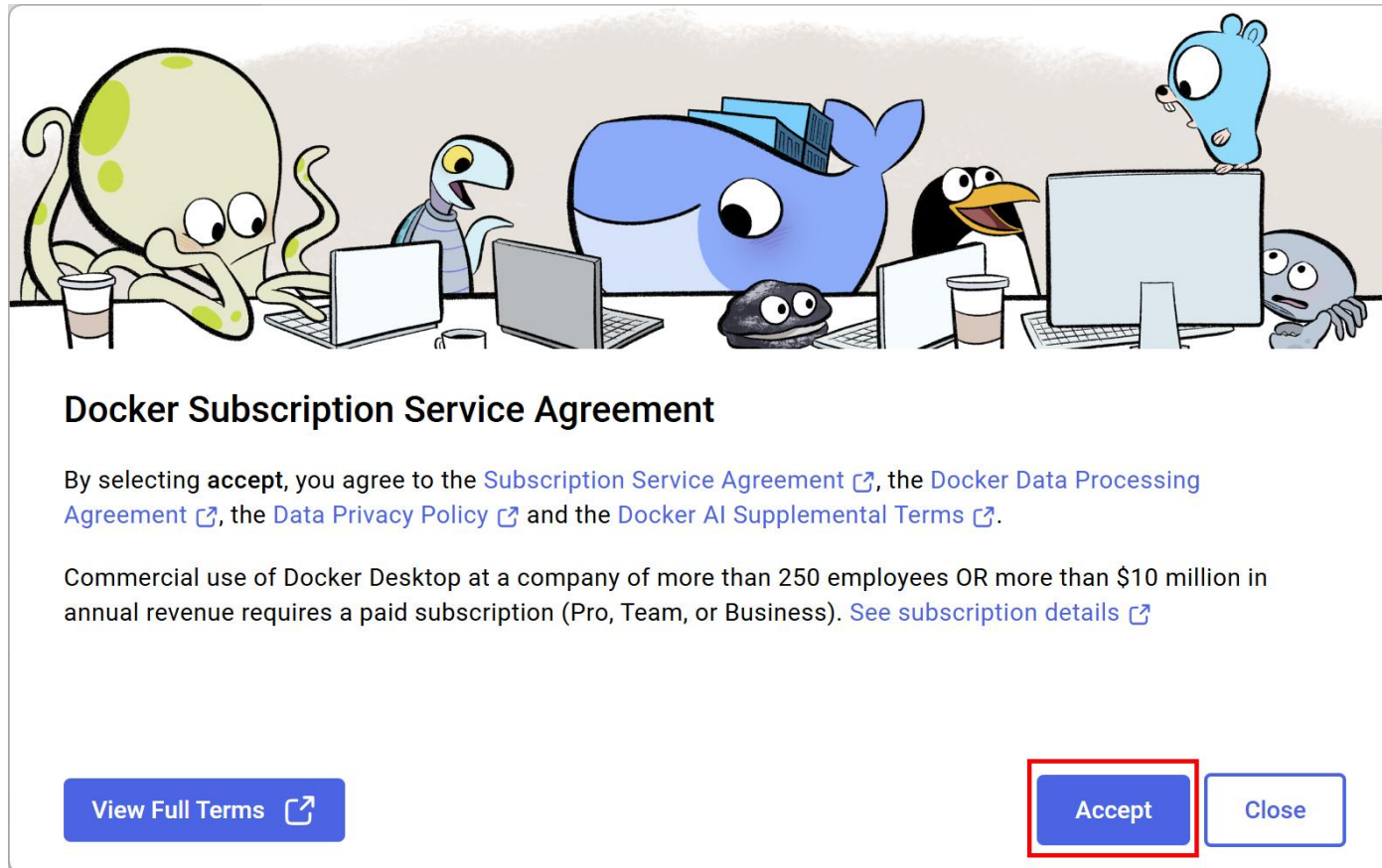


3.1 윈도우용 Docker 설치



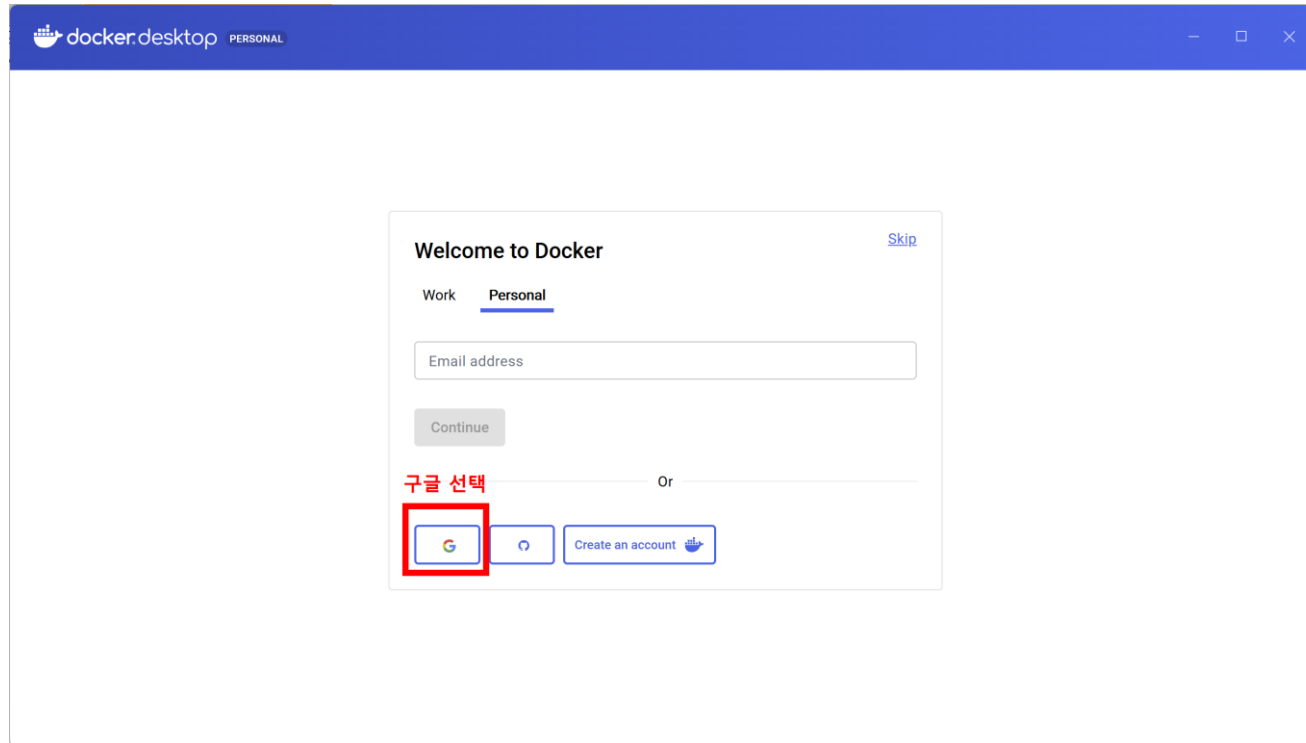
3.2 Docker Desktop 실행

Docker Desktop 실행



3.2 Docker Desktop 실행

Docker Desktop 실행



3.2 Docker Desktop 실행

Docker Desktop 실행

Create your username

Continue with your Google account or [choose another](#).



kyong yeol in (인경열)

inky4832@gmail.com

Username

kyongyeolin



Send me occasional product updates and announcements.

Sign up

By creating an account I agree to the [Subscription Service Agreement](#),
[Privacy Policy](#), [Data Processing Terms](#).



Verify your email

We've sent a 6-digit code to inky4832@gmail.com

This sign-up session will expire in 15 minutes.

Enter code

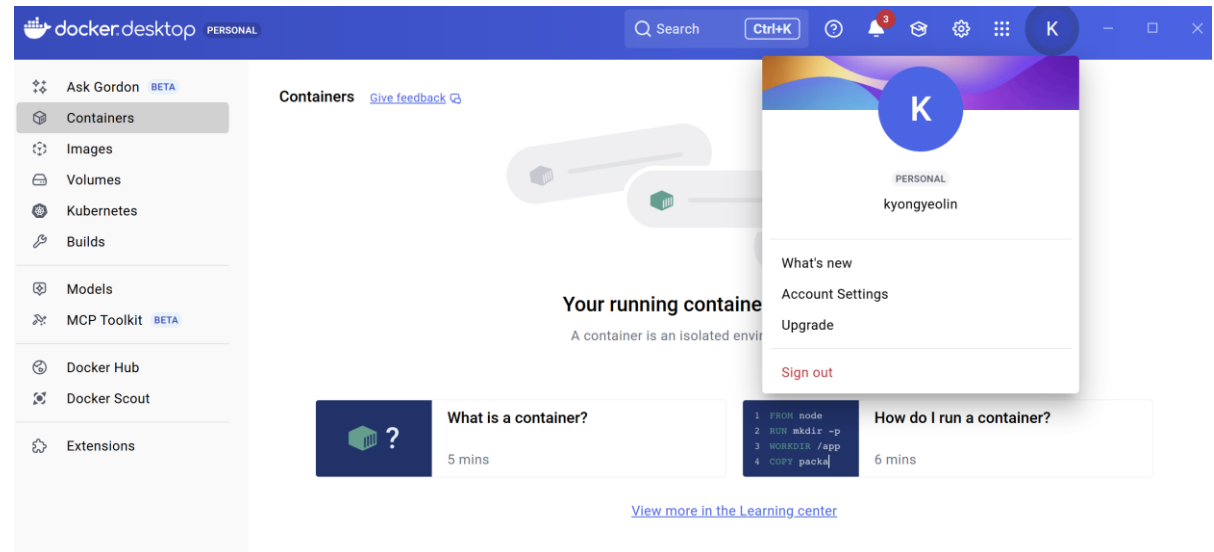
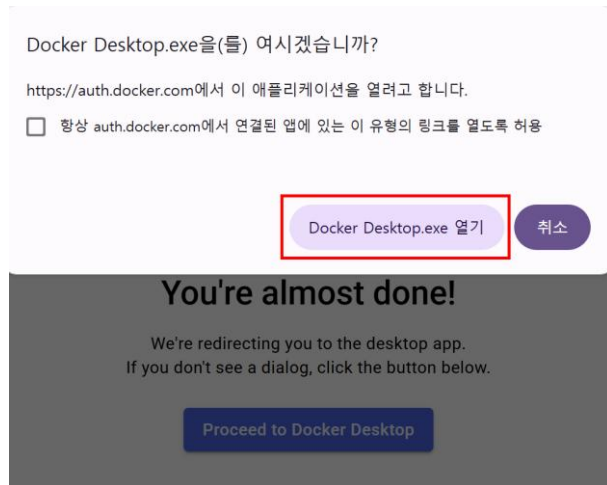
Verify

[Resend verification code](#)

[Change email or username](#)

3.2 Docker Desktop 실행

Docker Desktop 실행



3.2 Docker 설치 확인

Docker 버전 확인

- **docker version**

```
C:\docker_study>docker version
Client:
 Version:           29.1.3
 API version:       1.52
 Go version:        go1.25.5
 Git commit:        f52814d
 Built:             Fri Dec 12 14:51:52 2025
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.56.0 (214940)
Engine:
 Version:           29.1.3
 API version:       1.52 (minimum version 1.44)
 Go version:        go1.25.5
 Git commit:        fbf3ed2
 Built:             Fri Dec 12 14:49:51 2025
 OS/Arch:           linux/amd64
 Experimental:      false
containerd:
 Version:           v2.2.1
 GitCommit:        dea7da592f5d1d2b7755e3a161be07f43fad8f75
runc:
 Version:           1.3.4
 GitCommit:        v1.3.4-0-gd6d73eb8
docker-init:
 Version:           0.19.0
 GitCommit:        de40ad0
```

3.3 hello-world 실행

hello-world 컨테이너 실행

- **docker run hello-world**

```
C:\docker_study>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:
<https://hub.docker.com/>

For more examples and ideas, visit:
<https://docs.docker.com/get-started/>

내부 동작 순서

1. hello-world 이미지가 있는지 확인
2. 없으면 Registry (Docker hub)에서 다운로드
3. 이미지로 컨테이너 생성
4. 컨테이너 실행
5. 메시지 출력 후 종료

3.3 hello-world 실행

컨테이너 상태 확인

- `docker ps`
- `docker ps -a`

```
C:\docker_study>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
C:\docker_study>docker ps -a						
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
546f4733b88c	hello-world	"/hello"	3 minutes ago	Exited (0) 3 minutes ago		angry_hofstadter

다운로드 된 이미지 목록 확인

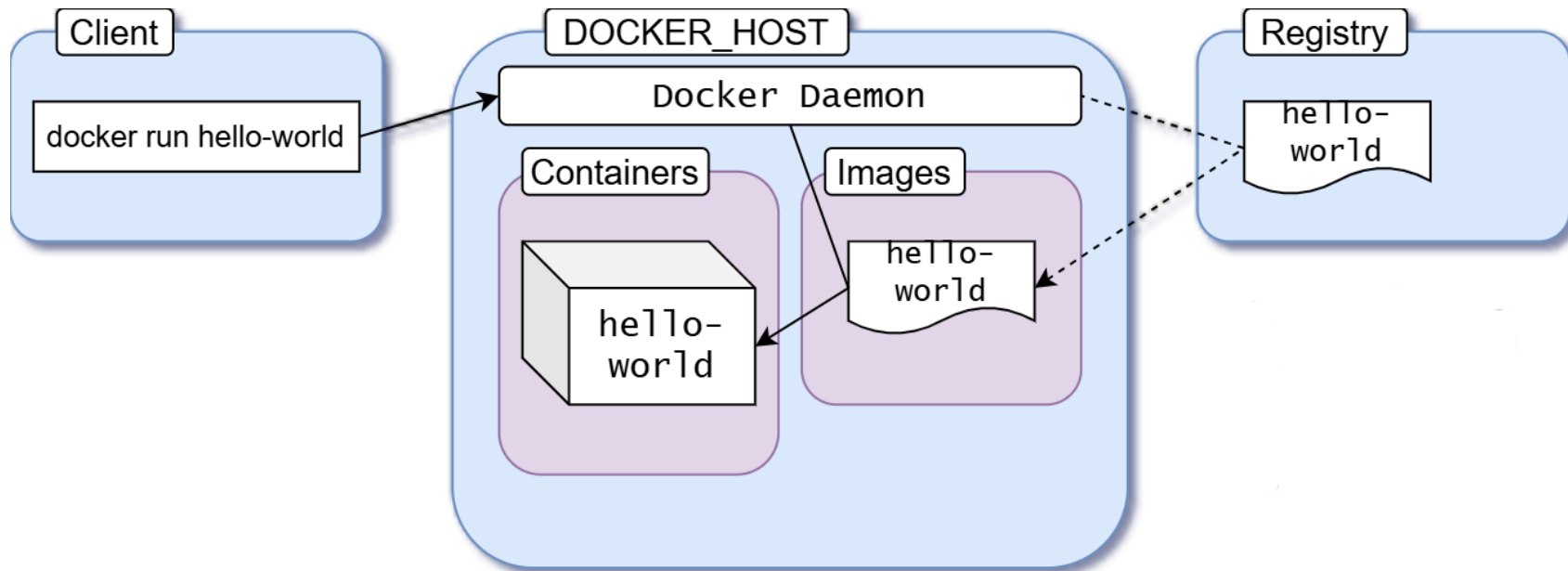
- `docker images`

```
C:\docker_study>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	f9078146db2e	25.9kB	9.49kB	U

3.3 hello-world 실행

hello-world 아키텍처



3.4 nginx 실행

nginx 컨테이너 실행

- **docker run -d nginx # background에서 실행**

```
C:\docker_study>docker run -d nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
ff048f1f2159: Pull complete
85c66128325a: Pull complete
801a1ad15b4e: Pull complete
3531af2bc2a9: Pull complete
4677c2a9a3d4: Pull complete
ce776bbcdad0: Pull complete
677c63196868: Pull complete
54c38c75806e: Download complete
69989ccd189b: Download complete
Digest: sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc
Status: Downloaded newer image for nginx:latest
b0d5f93d9b23f093782ba3df4d379c52800e0fcc4239eb2583b1df10f8eb7c61
```

```
C:\docker_study>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b0d5f93d9b23	nginx	"/docker-entrypoint...."	2 minutes ago	Up 2 minutes	80/tcp	happy_mahavira

3.4 nginx 실행

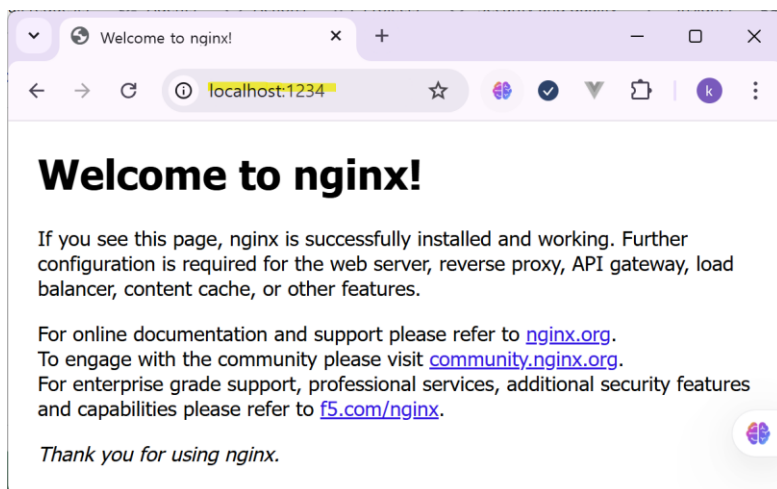
nginx 컨테이너 실행 + port forwarding

- `docker run -d -p 1234:80 nginx`

```
C:\docker_study>docker run -d -p1234:80 nginx
47c8fef9ec39ae293b5ed2df786c628d0d9f9f40d045037577163e51a29e0da7
```

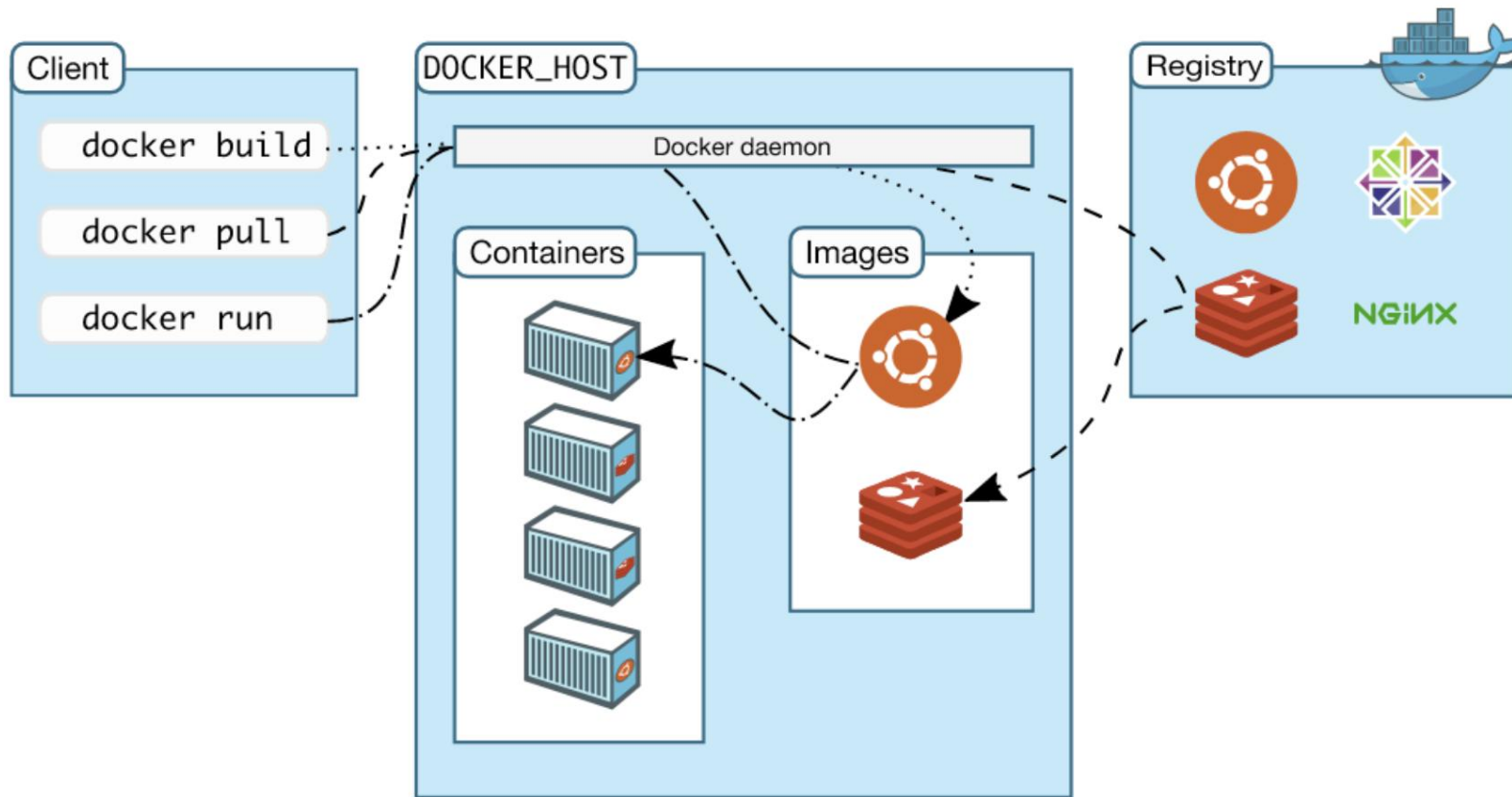
```
C:\docker_study>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
47c8fef9ec39	nginx	"/docker-entrypoint...."	12 seconds ago	Up 11 seconds	0.0.0.0:1234->80/tcp
b0d5f93d9b23	nginx	"/docker-entrypoint...."	5 minutes ago	Up 5 minutes	80/tcp



3.5 Docker Architecture

Docker 아키텍처



4.1 docker Image 명령어

image 명령어 개요

- docker image는 읽기 전용의 불변 형태로 만들어짐

명령어	설명
<code>docker image ls</code>	docker image 목록 보기, 별칭: <code>docker images</code>
<code>docker image inspect</code>	docker image 세부 정보 조회
<code>docker image history</code>	docker image 레이블 정보와 레이어 수행명령, 크기 정보 조회
<code>docker image tag</code>	원본 image에 새로운 참조명 설정, 별칭: <code>docker tag</code>
<code>docker image rm</code>	docker image 삭제, 실행 중이면 삭제 불가, 별칭: <code>docker rmi</code> , <code>docker image remove</code>
<code>docker image prune -a</code>	모든 docker image 삭제
<code>docker image save</code>	docker image를 파일로 저장
<code>docker image pull</code>	docker hub에서 로컬로 docker image 내려 받기. 별칭: <code>docker pull</code>
<code>docker image push</code>	로컬에 있는 docker image를 docker hub에 업로드하기, 별칭: <code>docker push</code>
<code>docker login</code>	업로드를 하기 전 docker hub 계정으로 로그인 수행하기
<code>docker logout</code>	docker hub에서 로그아웃 하기

4.1 docker Image 명령어

docker 이미지 목록 조회

- 기능: 로컬에 저장된 Docker 이미지 목록을 조회하는 명령어.
- 문법
 - `docker image ls [옵션] [REPOSITORY[:TAG]]`
 - 별칭: `docker images`
- 옵션
 - `-a` : 모든 이미지 표시
 - `-q` : 이미지 ID만 출력
 - `--no-trunc` : 이미지 ID를 줄이지 않고 전체 표시
 - `--digests` : 이미지 digest 표시
 - `-f` : 조건으로 필터링
 - `--format` : 출력 형식 지정

4.1 docker Image 명령어

docker 이미지 목록 조회

- 기능: 로컬에 저장된 Docker 이미지 목록을 조회하는 명령어.
- 문법
 - `docker image ls [옵션] [REPOSITORY[:TAG]]`
 - 별칭: `docker images`
- 옵션
 - `-a` : 모든 이미지 표시
 - `-q` : 이미지 ID만 출력
 - `--no-trunc` : 이미지 ID를 줄이지 않고 전체 표시
 - `--digests` : 이미지 digest 표시
 - `-f, --filter` : 조건으로 필터링
 - `--format` : 출력 형식 지정

ex. `docker images -f "before=nginx"`
`docker images --no-trunc`
`docker images --format json`

4.1 docker Image 명령어

docker 이미지 상세 조회

- 기능: Docker 이미지의 상세 메타데이터를 JSON 형식으로 출력.
- 문법
 - `docker image inspect [options] IMAGE [IMAGE ..]`
- 옵션
 - `-f, --format` : 원하는 정보만 출력가능, 대소문자 구별.

ex.

```
docker image inspect nginx
```

```
docker image inspect nginx -f '{{.Id}}'
```

4.1 docker Image 명령어

docker 이미지 Layer 조회

- 기능: Docker 이미지가 어떤 Layer들로 구성되어 있는지 조회.
- 문법
 - **docker image history [OPTIONS] IMAGE**
 - 별칭: **docker history**
- 옵션
 - --format : 원하는 정보만 출력가능, 대소문자 구별.
 - --no-trunc : 이미지 ID를 줄이지 않고 전체 표시
 - -q : 이미지 ID만 출력

ex.

```
docker image history nginx
```

```
docker image history nginx --format '{{.ID}}'
```

4.1 docker Image 명령어

```
C:\docker_study>docker image history nginx
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
6e23479198b9	3 days ago	CMD ["nginx" "-g" "daemon off;"]	0B	buildkit.dockerfile.v0
<missing>	3 days ago	STOPSIGNAL SIGQUIT	0B	buildkit.dockerfile.v0
<missing>	3 days ago	EXPOSE map[80/tcp:{}]	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENTRYPOINT ["/docker-entrypoint.sh"]	0B	buildkit.dockerfile.v0
<missing>	3 days ago	COPY 30-tune-worker-processes.sh /docker-ent...	16.4kB	buildkit.dockerfile.v0
<missing>	3 days ago	COPY 20-envsubst-on-templates.sh /docker-ent...	12.3kB	buildkit.dockerfile.v0
<missing>	3 days ago	COPY 15-local-resolvers.envsh /docker-entryp...	12.3kB	buildkit.dockerfile.v0
<missing>	3 days ago	COPY 10-listen-on-ipv6-by-default.sh /docker...	12.3kB	buildkit.dockerfile.v0
<missing>	3 days ago	COPY docker-entrypoint.sh / # buildkit	8.19kB	buildkit.dockerfile.v0
<missing>	3 days ago	RUN /bin/sh -c set -x && groupadd --syst...	86.7MB	buildkit.dockerfile.v0
<missing>	3 days ago	ENV DYNPKG_RELEASE=1~trixie	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV PKG_RELEASE=1~trixie	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV ACME_VERSION=0.3.1	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV NJS_RELEASE=1~trixie	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV NJS_VERSION=0.9.6	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV NGINX_VERSION=1.29.8	0B	buildkit.dockerfile.v0
<missing>	3 days ago	LABEL maintainer=NGINX Docker Maintainers <d...	0B	buildkit.dockerfile.v0
<missing>	4 days ago	# debian.sh --arch 'amd64' out/ 'trixie' '@1...	87.4MB	debuerreotype 0.17

4.1 docker Image 명령어

docker 이미지에 TAG 설정

- 기능: 기존 이미지에 새로운 이름(태그)을 부여하는 명령어.
- 문법
 - `docker image tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]`
 - 별칭: `docker tag`
- 특징
 - TAG 생략 시 자동으로 latest 지정 (비추천)
 - latest라고해서 가장 최신버전이 아님.
- 용도: 버전 관리용

ex. `docker tag nginx:latest nginx:v1`
`docker tag nginx nginx:v2`
`docker tag nginx nginx:test`

4.1 docker Image 명령어

docker 이미지 삭제

- 기능: 지정된 이미지에 해당하는 Docker 이미지 삭제
컨테이너 실행 중에는 삭제 불가
- 문법
 - **docker image rm [OPTIONS] IMAGE [IMAGE...]**
 - 별칭: docker image remove , docker rmi
- 옵션: -f, --force : 강제 삭제

ex. docker image rm 5aca99593157
docker image rm nginx
docker image rm nginx:v1
docker image rm nginx mysql

4.1 docker Image 명령어

docker 이미지 파일로 저장

- 기능: Docker 이미지를 tar 파일로 저장하는 명령어.
- 문법
 - `docker image save [OPTIONS] IMAGE [IMAGE...]`
 - 별칭: `docker save`
- 옵션: `-o`, `--output` : stdout 대신에 파일에 쓰기

ex.

```
docker save nginx -o nginx.tar
```

```
docker save nginx > nginx.tar
```

```
docker load -i nginx.tar
```

4.1 docker Image 명령어

docker 이미지 다운로드

- 기능: docker hub에서 로컬로 docker image를 다운로드함.
- 문법
 - `docker image pull [OPTIONS] NAME[:TAG|@DIGEST]`
 - 별칭: `docker pull`
- 옵션: `-a, --all-tags` : 저장소에 태그로 지정된 이미지를 모두 다운로드 함.
`-q, --quiet` : 이미지 다운로드 과정에서 상세출력 숨김.

ex.

```
docker pull ubuntu
```

```
docker pull ubuntu:24.04
```

```
docker pull ubuntu@sha256:c4a8d5503dfb2a # TAG 대신에 Digest 이용
```

4.1 docker Image 명령어

docker 이미지 업로드

- 기능: 로컬에 있는 docker image를 docker hub에 업로드하기
- 문법
 - `docker image push [OPTIONS] NAME[:TAG]`
 - 별칭: `docker push`

실습순서

- 1) `https://hub.docker.com` 에서 repository 생성 (ex. my-app-repo)
- 2) 로컬에서 `docker login -u 이메일 로그인`
- 3) 이미지 태깅
`docker tag nginx:latest 계정명/my-app-repo:tag`
- 4) 이미지 push
`docker push 계정명/my-app-repo:tag`

4.1 docker Image 명령어

[Repositories](#) / [my-app-repo](#) / [General](#)

kyongyeolin/my-app-repo

Last pushed 10 minutes ago • Repository size: 60.2 MB • ☆0 • ↓8

my-app-repo 

Add a category  

Docker commands

To push a new tag to this repository:

```
docker push kyongyeolin/my-app-repo:tagname
```

Public view

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags

 DOCKER SCOUT INACTIVE [Activate](#)

This repository contains 0 tag(s).

Tag	OS	Type	Pulled	Pushed
 latest		Image	less than 1 day	10 minutes
 v1		Image	less than 1 day	12 minutes



Build with
Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

4.1 docker Image 명령어

docker login / logout

- 기능: docker hub 계정으로 로그인 및 로그아웃 하기
- 문법
 - `docker login -u <계정>`
 - `docker logout`

```
C:\docker_study>docker login -u inky4832@gmail.com
```

i Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit <https://app.docker.com/settings>

Password:

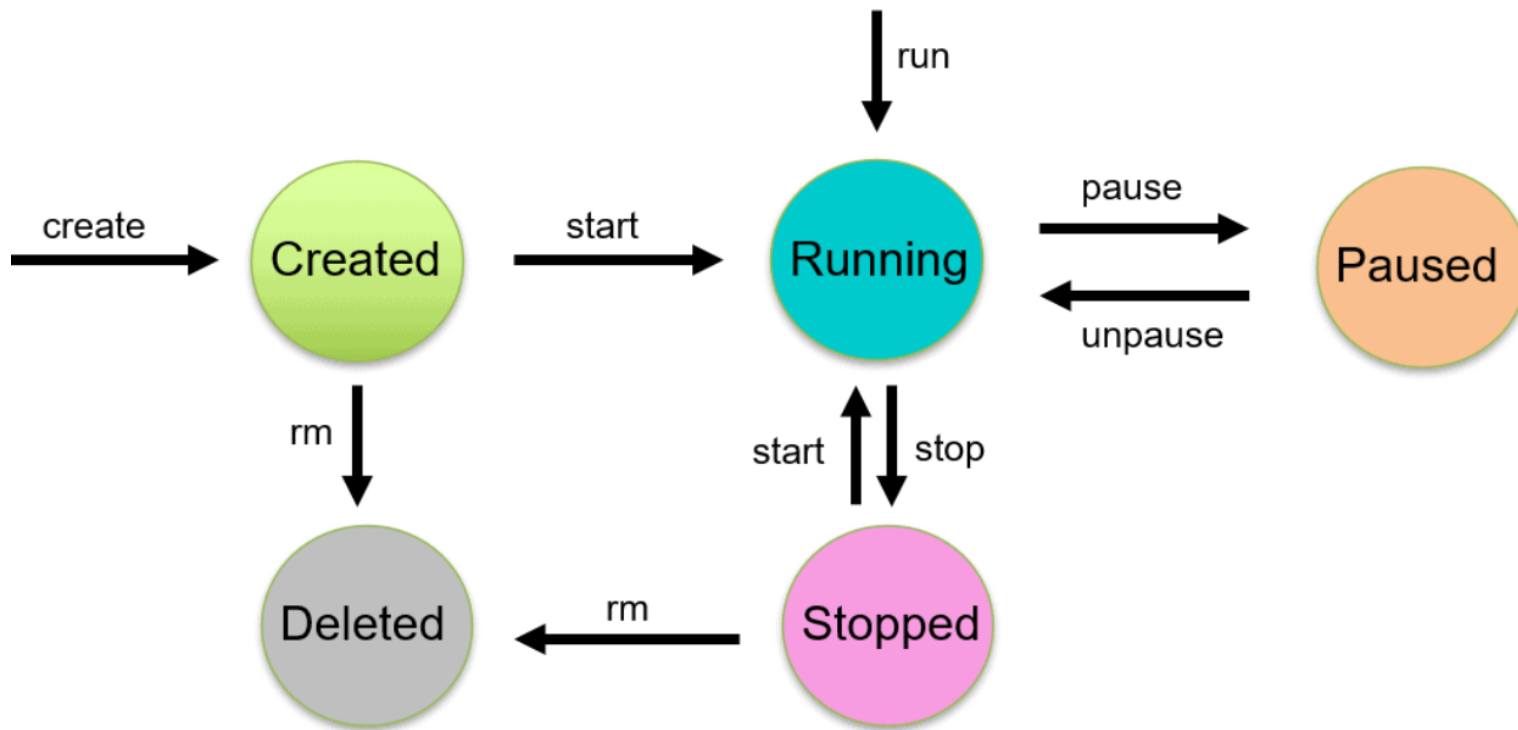
Login Succeeded

```
C:\docker_study>docker search httpd
```

NAME	DESCRIPTION
httpd	The Apache HTTP Server Project
manageiq/httpd	Container with httpd, built on CentOS
paketobuildpacks/httpd	

4.2 docker Container 명령어

Docker Container LifeCycle 관리



4.2 docker Container 명령어

container 명령어

- docker image를 이용하여 Container를 생성

명령어	설명
docker container create	컨테이너 생성 (실행은 안함), 별칭: docker create
docker container start	이미 존재하는(생성된) 컨테이너를 실행 상태로 시작, 별칭: docker start
docker container stop	컨테이너를 정상 종료 (graceful shutdown) 시킴, 별칭: docker stop
docker container restart	컨테이너를 재시작 (stop + start), 별칭: docker restart
docker container pause	컨테이너를 일시 정지 시킴, 별칭: docker pause
docker container unpause	pause로 멈춘 컨테이너를 다시 재개(resume), 별칭: docker unpause
docker container run	image로부터 컨테이너를 만들고(create) 즉시 실행(start)까지 한 번에 동작됨 별칭: docker run
docker container ls	현재 실행 중인 컨테이너 목록 별칭: docker ps
docker exec	실행중인 컨테이너 내부에서 명령을 추가로 실행
docker cp	호스트와 컨테이너 간 파일 복사
docker inspect	컨테이너 상세 정보
docker rm	컨테이너 완전히 삭제 , 실행 중인 컨테이너는 삭제 불가

4.2 docker Container 명령어

docker 컨테이너 생성

- 기능: 컨테이너를 생성하는 명령어 (실행 안함)
이미지가 없으면 자동으로 다운로드 됨
- 문법
 - **docker container create [OPTIONS] IMAGE [COMMAND] [ARG...]**
별칭: docker create
- 옵션: --name : 컨테이너 이름
지정하지 않으면 임의의 값으로 자동설정.
 - p : 포트 매핑
 - e, --env : 환경변수 지정
 - v : 볼륨 지정
 - network: 네트워크 지정

4.2 docker Container 명령어

ex.

```
docker create --name webserver nginx
```

```
docker create -p 8080:80 nginx
```

```
docker create -e MYSQL_ROOT_PASSWORD=1234 mysql:8.3
```

```
docker create -v mydata:/var/lib/mysql mysql:8.3
```

```
docker create --network fullstack-net nginx
```

```
docker ps -a
```

4.2 docker Container 명령어

docker 컨테이너 시작

- 기능: 중지(Stopped) 되었거나 Created 상태인 컨테이너를 실행(Start)하는 명령어.
기본적으로 background 방식으로 시작됨.
- 문법
 - **docker container start [OPTIONS] CONTAINER [CONTAINER...]**
별칭: docker start
- 옵션: -a : attach (foreground) 방식 지정

ex.

```
docker ps -a
```

```
docker start ce0875e0b809
```

4.2 docker Container 명령어

docker 컨테이너 종료

- 기능: 실행(Running)중인 컨테이너를 정상적으로 종료(Graceful Shutdown)하는 명령어
- 문법
 - **docker container stop [OPTIONS] CONTAINER [CONTAINER...]**
별칭: docker stop
- 옵션: -s, --signal : 컨테이너에 보내는 시그널
-t, --timeout : 컨테이너를 종료하기 전 wait 시간. (window : 30초 기본)

ex.

```
docker ps
```

```
docker stop ce0875e0b809
```

```
docker stop ce0875e0b809 nginx
```

```
docker stop $(docker ps -q) # 한꺼번에 종료
```

```
docker start ce0875e0b809 # 다시 시작
```

4.2 docker Container 명령어

docker run

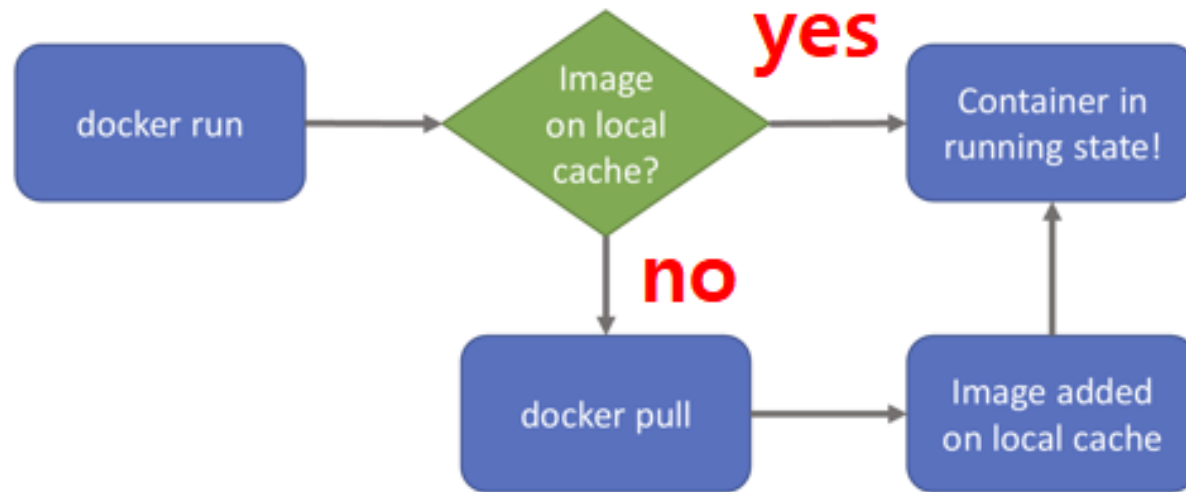
- 기능: image로부터 컨테이너를 생성하고 즉시 실행까지 한 번에 동작
([pull] + create + start)
- 문법
 - **docker container run [OPTIONS] IMAGE [COMMAND] [ARG...]**

별칭: docker run

주요 옵션	설명
--name	컨테이너 이름 지정
-d	백그라운드 실행 (detached)
-a	포그라운드 실행 (attach), 기본
-p 호스트포트:컨테이너포트	포트 포워딩
-v 호스트경로:컨테이너경로	볼륨/바인딩 마운트
-e key=value	환경 변수
--rm	컨테이너 종료 시 자동 삭제
-it	터미널 입력 가능 (대화형)
--network	여러 컨테이너들을 동일 네트워크로 묶을 때 사용

4.2 docker Container 명령어

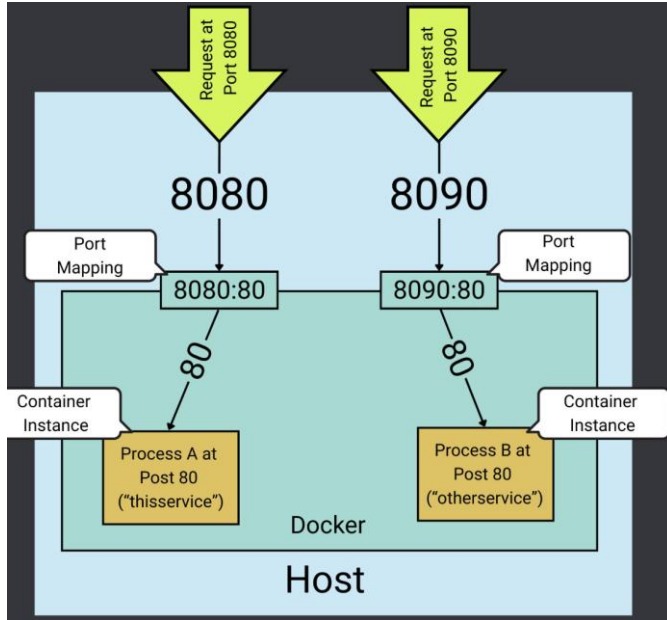
docker run



4.2 docker Container 명령어

port forwarding

- 개념: 호스트의 포트를 컨테이너 내부 포트로 트래픽을 전달하는 기능.
 - 문법
 - `-p [호스트포트]:[컨테이너포트]`
- ex. `docker run -d -p 8080:80 nginx`



4.2 docker Container 명령어

ex.

```
docker run hello-world
```

```
docker run -it --name ubuntu1 ubuntu:24.04 bash
```

```
docker run -d --name web3 -p 8080:80 -e APP_NAME=myapp nginx
```

```
docker run -d --name web4 -p 8090:80 --network fullstack-net2 nginx
```

```
docker run -d --name web5 -p 8091:80 --rm nginx
```

4.2 docker Container 명령어

docker 컨테이너 목록 조회

- 기능: 현재 실행 (Running) 중인 컨테이너 목록을 조회하는 명령어
- 문법
 - **docker container ls [OPTIONS]**
별칭: docker container list, docker container ps, **docker ps**
- 옵션:
 - a : 모든 컨테이너 조회 (종료 포함)
 - q : 컨테이너 ID만 출력.
 - l, --latest : 가장 최근에 생성된 컨테이너 1개 출력
 - n, --last : 최근 N개 컨테이너 조회
 - s : 컨테이너 크기
 - filter : 조건 검색
 - format: 원하는 형식으로 출력

4.2 docker Container 명령어

ex.

```
docker ps
```

```
docker ps -a
```

```
docker ps -q
```

```
docker ps -aq
```

```
docker ps -l
```

```
docker ps -n 5
```

```
docker ps --filter status=exited
```

```
docker ps --filter name=web
```

```
docker ps --format "{{.Names}}"
```

```
docker ps -q | wc -l
```

4.2 docker Container 명령어

docker 컨테이너에서 새로운 프로세스 실행

- 기능: 실행 중인 컨테이너 내부에서 새로운 프로세스를 실행하는 명령어
- 문법
 - `docker container exec [OPTIONS] CONTAINER COMMAND [ARG...]`
 - 별칭: `docker exec`
- 옵션:
 - i : interactive, 표준입력 유지
 - t : tty 생성
 - e : 환경 변수
 - d : detach (백그라운드)
 - w : working dir 지정

4.2 docker Container 명령어

ex.

```
docker exec web1 pwd
```

```
docker exec -w /etc/nginx web1 pwd
```

```
docker exec web1 env
```

```
docker exec web1 ps -ef
```

```
docker exec web1 cat /etc/nginx/nginx.conf
```

```
docker exec -it redis-server redis-cli
```

```
docker exec -it ubuntu1 bash
```

```
docker exec -it web1 sh
```

```
docker exec -it mysql-db mysql -u root -p
```

4.2 docker Container 명령어

docker 컨테이너와 호스트 간 파일/디렉터리 복사

- 기능: 호스트와 컨테이너간 파일/디렉터리 복사하는 명령어
- 문법
 - `docker container cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH`
 - `docker cp [OPTIONS] SRC_PATH CONTAINER:DEST_PATH`
 - 별칭: `docker cp`
- 옵션: `-a, --archive` : 파일 소유권 유지
 `-L, --follow-link` : 심볼릭 링크 원본 복사

ex.

```
docker cp test.txt ubuntu1:/root/
```

```
docker cp ubuntu1:/root/sample.txt .
```

4.2 docker Container 명령어

docker 컨테이너 상세 정보

- 기능: 컨테이너, 이미지, 네트워크, 볼륨 등 상세 정보를 JSON 형식으로 조회하는 명령어
- 문법
 - `docker container inspect [OPTIONS] CONTAINER [CONTAINER...]`
 - 별칭: `docker inspect`
- 옵션: `-f, --format` : 특정 정보만 출력
`-s, --size` : total 파일 사이즈

ex.

```
docker inspect web1 ubuntu1
```

```
docker inspect -f '{{.State.Status}}' web1
```

```
docker inspect -f '{{.Config.Image}}' web1
```

4.2 docker Container 명령어

docker 컨테이너 삭제

- 기능: 컨테이너를 삭제하는 명령어
- 문법
 - `docker container rm [OPTIONS] CONTAINER [CONTAINER...]`
 - 별칭: `docker container remove`, `docker rm`
- 옵션: `-f, --force` : 강제 삭제
`-v, --volumes` : 볼륨 삭제

ex.

```
docker rm web1
```

```
docker rm -f web1
```

```
docker rm $(docker ps -aq) # 모든 컨테이너 삭제
```

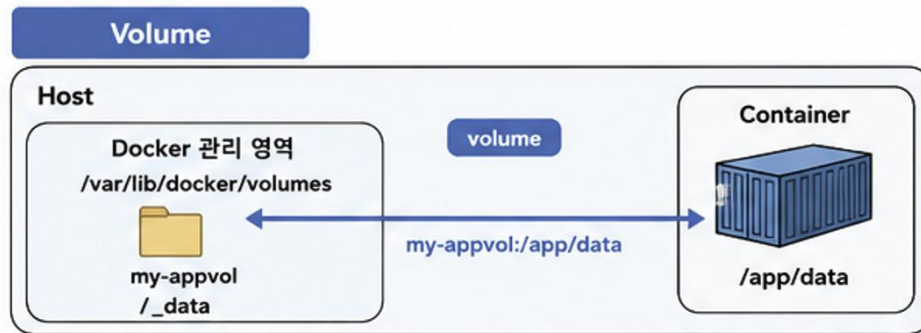
5.1 Docker 사용 시 데이터 종류

docker에서 사용되는 데이터 종류

- **image 에 저장되는 데이터**
 - 이미지는 컨테이너 실행에 필요한 모든 정보를 저장하는 템플릿.
 - 저장되는 내용: OS라이브러리, JDK등의 실행환경, app코드, 설정파일, 의존성 파일 등
 - 특징: Read-Only, 여러 컨테이너가 하나의 이미지 공유가능
- **container 에 저장되는 데이터**
 - 실행 중 생성되는 데이터는 컨테이너 Layer에 저장됨.
 - 저장되는 내용: 실행 중 생성된 파일, 로그 파일, 업로드 파일 등
 - 특징: Read-Write, 컨테이너마다 독립적, 휘발성(Emphemeral)
- **볼륨(volume) 에 저장되는 데이터**
 - 컨테이너가 삭제되어도 유지되어야 하는 데이터를 저장하기 위한 공간임.
 - 저장되는 내용: MySQL 데이터, 업로드 파일, 로그 파일 등
 - 특징: 영구 저장(Persistent), 여러 컨테이너가 공유 가능.

5.2 영속성 데이터 관리 방법

볼륨 (volume)



샘플 예제

1) Volume 사용 예제 (MySQL 데이터 영구 저장)

```
# 1. 볼륨 생성
docker volume create mysql-data

# 2. MySQL 컨테이너 실행
docker run -d \
  --name mysql \
  -e MYSQL_ROOT_PASSWORD=1234 \
  -v mysql-data:/var/lib/mysql \
  -p 3306:3306 \
  mysql:8.3
```

구조

```
/var/lib/docker/volumes
├── mysql-data
│   └── _data
│       └── ibdata1, ... (DB 파일)
```

```
Container (mysql)
/var/lib/mysql
```

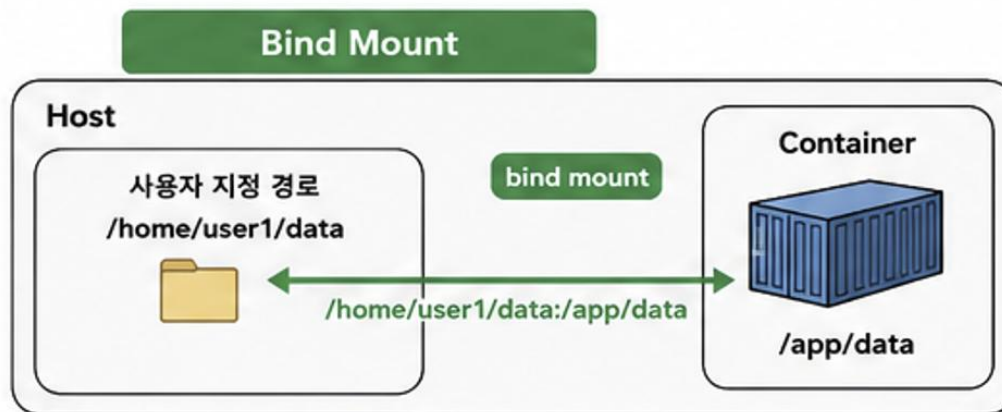
확인

```
docker volume ls
docker volume inspect mysql-data
```

구분	Volume
관리 주체	Docker가 관리
경로 예시	my-appvol:/var/log
저장 위치	/var/lib/docker/volumes/<볼륨이름>/_data
생성 방법	docker volume create <이름>
사용 목적	DB 데이터, 영속 데이터
이식성	우수 (다른 서버/환경에서도 동일하게 사용 가능)
성능	Docker 내부에서 관리하므로 안정적
사용 빈도	운영 환경에서 주로 사용 (DB 등)

5.2 영속성 데이터 관리 방법

바인드 마운트 (Bind Mount)



2) Bind Mount 사용 예제 (소스 코드 실시간 반영)

```
# 현재 디렉터리의 코드를 컨테이너 /app에 연결
docker run -d \
  --name app \
  -v $(pwd):/app \
  -p 8080:8080 \
  openjdk:21-jdk \
  bash -c "cd /app && ./gradlew bootRun"
```

확인

```
# host에서 파일 수정 -> 컨테이너에 즉시 반영
vim src/main/java/.../HelloController.java
```

구조

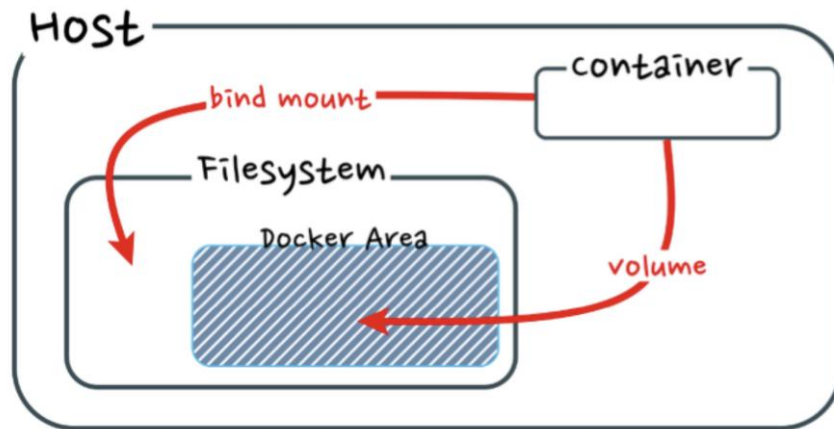
```
Host
/home/user1/my-project
├ src/
├ build.gradle
└ ...
```

```
Container (app)
/app
```

구분	Volume
관리 주체	사용자가 직접 host에 폴더 만들고 관리
경로 예시	<code>/home/user1/data:/var/log</code>
저장 위치	사용자가 지정한 실제 경로
생성 방법	host 경로만 있으면 됨
사용 목적	개발 소스코드, 설정파일, 로그 등
이식성	낮음 (경로는 서버마다 다를 수 있음)
성능	host 파일 시스템을 직접 사용 (I/O 성능은 host에 영향)
사용 빈도	개발 환경에서 주로 사용 (소스코드 등)

5.2 영속성 데이터 관리 방법

볼륨 (volume) 과 바인드 마운트(bind mount)



구분	Volume	Bind Mount
관리 주체	Docker가 경로 관리	사용자가 직접 host에 폴더 만들고 경로 관리
경로	my-appvol:/var/log	/home/user1/data:/var/log
추천 용도	DB 데이터, 운영 데이터	개발 소스 연결
이식성	좋음	호스트 실제 경로에 의존 (비권장)

5.3 Docker Volume

볼륨 (volume) 명령어

- 개념

- 컨테이너 데이터를 영구적으로 저장하기 위한 Docker가 관리하는 저장소.
- docker가 내부에서 관리함.

명령어	설명
<code>docker volume create 볼륨명</code>	volume 생성
<code>docker volume ls</code>	volume 목록 조회
<code>docker volume inspect 볼륨명</code>	volume 상세 정보
<code>docker rm -f 컨테이너명</code> <code>docker volume rm 볼륨명</code>	volume 삭제 (먼저 연결된 컨테이너를 삭제)
<code>docker volume prune</code>	사용하지 않는 volume 모두 삭제
<code>docker run -v 볼륨명:컨테이너경로 이미지</code>	volume 연결
<code>docker run --mount source=볼륨명, target=컨테이너경로</code>	volume 연결

5.3 Docker Volume

실습1 - nginx 와 Volume

#1. Volume 생성

```
docker volume create web-data
```

```
docker volume ls
```

```
docker volume inspect web-data
```

#2. web 컨테이너 생성

```
docker run -it -d --name web -p 1234:80 -v web-data:/usr/share/nginx/html nginx
```

#3. 컨테이너에 접속

```
docker exec -it web bash
```

#4. 파일 생성

```
echo "Hello Volume" > /usr/share/nginx/html/index.html
```

http://localhost:1234/ 요청

5.3 Docker Volume

실습1 - nginx 와 Volume

#5. 컨테이너 삭제

```
docker rm -f web
```

```
docker ps -a
```

#6. 동일 Volume 재사용하여 web2 컨테이너 생성

```
docker run -it -d --name web2 -p 1234:80 -v web-data:/usr/share/nginx/html nginx
```

#7. 웹브라우저 요청

5.3 Docker Volume

실습2 - nginx 와 Bind Mount

#1. Host에 직접 경로 생성

```
mkdir testdir
```

```
echo "Hello Host" > testdir/index.html
```

#2. web 컨테이너 생성

```
docker run -it -d --name nginx-bind -p 1235:80
```

```
-v C:\cloud_engineering\Docker/testdir:/usr/share/nginx/html nginx
```

http://localhost:1235 요청

#3. 컨테이너 삭제

```
docker rm -f nginx-bind
```

```
docker ps -a
```

5.3 Docker Volume

실습2 - nginx 와 Bind Mount

#4. 동일 Host 경로 재사용하여 nginx-bind2 컨테이너 생성

```
docker run -it -d --name nginx-bind -p 1235:80 -v C:\cloud_engineering\Docker/testdir:/usr/share/nginx/html nginx
```

#5. 웹브라우저 요청

`http://localhost:1235`

#6. Host 경로 파일 수정

```
echo "Modified" > testdir/index.html
```

#7. 웹브라우저 새로고침 하면 수정사항 바로 반영됨.

5.2 Docker 환경변수

env (환경변수)

- 개념

- 컨테이너 실행 시 외부에서 값을 주입해서 어플리케이션 설정을 변경하는 핵심방법.
- 코드 수정없이 DB 접속 정보, Port, 모드(dev/prod)등을 변경할 수 있음.

- 문법

- -e 옵션 이용
 - 실행 시점에 주입
 - image 수정 없이 설정 변경 가능
 - 우선 순위가 가장 높음
 - ex. `docker run -e key=value`
- --env-file 옵션 이용
 - 대량 변수 관리 가능
 - .env 파일안에 환경 변수 설정
 - ex. `docker run --env-file .env`

5.2 Docker 환경변수

실습1 -e 옵션

단일 환경변수

- 1) `docker run -d --rm --name my-app -e MY_NAME=kyongyeolin nginx`
- 2) `docker exec -it my-app bash`
`root# env`
`root# echo $MY_NAME`

여러 환경변수

- 1) `docker run -it --rm --name my-app2 \`
 `-e DB_HOST=localhost \`
 `-e DB_PORT=3306 \`
 `-e MODE=prod nginx bash`
- 2) `root# env`
 `root# echo $DB_HOST`

5.2 Docker 환경변수

실습2 .env 파일 이용

1) .env 파일 작성

```
DB_HOST=localhost
```

```
DB_USER=kyin
```

```
DB_PASSWORD=password
```

2) `docker run -d --rm --name my-app2 --env-file .env nginx`

3) `docker exec -it my-app2 bash`

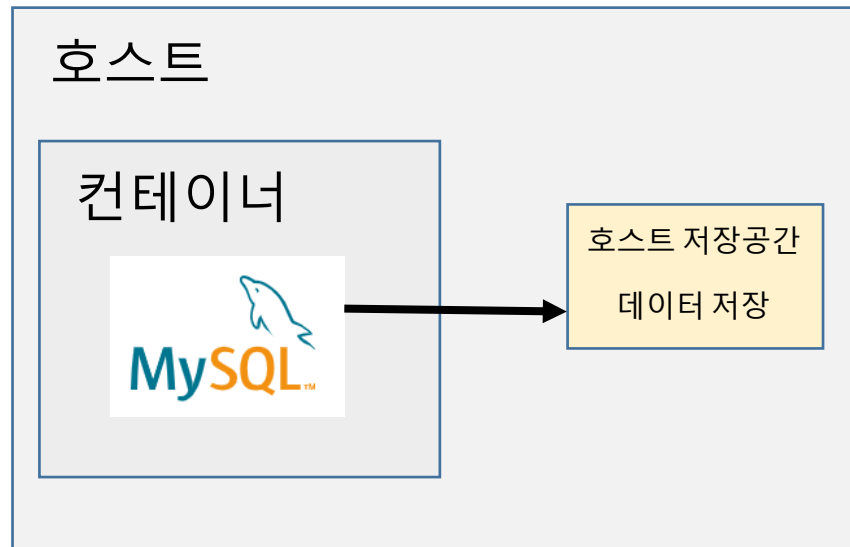
```
root# env
```

```
root# echo $DB_PASSWORD
```

5.3 MySQL 데이터베이스 설정

실습3 MySQL 데이터베이스 (볼륨 + env)

https://hub.docker.com/_/mysql#environment-variables



Environment Variables

When you start the `mysql` image, you can adjust the configuration of the MySQL in note that none of the variables below will have any effect if you start the container with `MYSQL_HOST` untouched on container startup.

See also <https://dev.mysql.com/doc/refman/5.7/en/environment-variables.html> for `MYSQL_HOST`, which is known to cause issues when used with this image).

MYSQL_ROOT_PASSWORD

This variable is mandatory and specifies the password that will be set for the MySQL

MYSQL_DATABASE

This variable is optional and allows you to specify the name of a database to be created for the superuser access ([corresponding to GRANT ALL](#)) to this database.

MYSQL_USER , MYSQL_PASSWORD

```
-v /my/own/datadir:/var/lib/mysql
```

5.3 MySQL 데이터베이스 설정

실습3 MySQL 데이터베이스 (볼륨 + env)

```
docker volume create mysql-data-vol
```

```
docker volume ls
```

```
docker run -it -d --name mysql-vtest -e MYSQL_ROOT_PASSWORD=1234 -e  
MYSQL_DATABASE=dockertest -v mysql-data-vol:/var/lib/mysql mysql:8.3
```

```
docker exec -it mysql-vtest bash
```

```
bash-4.2# mysql -u root -p
```

```
Enter password:
```

```
mysql> show databases;
```

5.3 MySQL 데이터베이스 설정

실습3 MySQL 데이터베이스 (볼륨 + env)

```
mysql> use dockertest;  
Database changed  
mysql> create table mytest ( name char, age int );  
Query OK, 0 rows affected (0.04 sec)  
mysql> insert into mytest (name, age) values ('a', 20);  
Query OK, 1 row affected (0.01 sec)  
mysql> exit  
bash-4.2# exit  
exit
```

```
docker stop mysql-vtest  
docker rm mysql-vtest
```

```
docker run -it -d --name mysql-vtest -e MYSQL_ROOT_PASSWORD=1234 -e  
MYSQL_DATABASE=dockertest -v mysql-data-vol:/var/lib/mysql mysql:8.3
```

5.3 MySQL 데이터베이스 설정

실습3 MySQL 데이터베이스 (볼륨 + env)

```
docker exec -it mysql-vtest bash
```

```
bash-4.2# mysql -u root -p
```

```
Enter password:
```

```
mysql> show databases;
```

```
mysql> use dockertest;
```

```
mysql> show tables;
```

```
+-----+
```

```
| Tables_in_dockertest |
```

```
+-----+
```

```
| mytest      |
```

```
+-----+
```

```
1 row in set (0.00 sec)
```

```
docker stop mysql-vtest
```

```
docker rm -f mysql-vtest
```

```
docker volume rm mysql-data-vol
```

6.1 Dockerfile

laC (Infrastructure as Code)

- 개념

- 서버, 네트워크, 로드밸런서, 보안그룹 같은 인프라를 코드로 정의하고 자동 생성하는 방식.
- 사용자의 개입없이 코드 실행만으로 인프라를 구축할 수 있음.

- 등장배경

- 기존 방식
 - 누가 어떻게 만들었는지 기록이 없음
 - 재현 불가능
 - 환경마다 설정이 모두 다름
 - 장애발생 시 원인 추적이 어려움
- laC 방식
 - 인프라가 코드로 남음
 - Git으로 버전 관리
 - 동일한 환경을 언제든지 재생성 가능
 - 자동화 & 표준화

6.1 Dockerfile

Dockerfile

- 개념
 - docker image를 생성할 때 사용하는 파일
 - <https://docs.docker.com/reference/dockerfile/>
- Dockerfile 기본 샘플

```
FROM ubuntu:22.04

RUN apt update
RUN apt install -y nginx

CMD ["nginx", "-g", "daemon off;"]
```

```
FROM ubuntu:22.04
WORKDIR /app
COPY . .
RUN apt update && apt install -y python3
CMD ["python3", "app.py"]
```


6.2 주요 명령어

Dockerfile 주요 명령어

- Dockerfile은 필요한 개발환경을 제공하기 위한 여러 가지 명령어들의 집합체임.
- 각 명령어의 의미와 권장 사용 방법을 통해 최적의 인프라 환경을 제공함.

명령어	설명
FROM	(필수) 생성하려는 이미지의 베이스 이미지 지정. Official image 권장. ex. FROM ubuntu:20.04
MAINTAINER	이미지를 빌드한 작성자 이름과 이메일을 작성. ex. MAINTAINER username <userid@example.com>
LABEL	이미지 작성 목적으로 버전, 타이틀, 설명, 라이선스 정보 등을 작성 ex. LABEL purpose = " version = '1.0' description = "
RUN	설정된 기본 이미지에 패키지 설치, 명령 실행 등을 작성. (이미지가 만들어질 때 한번만 실행됨) 실행 결과가 레이어로 저장됨. ex. RUN apt update && RUN apt install nginx -y
COPY	호스트 환경의 파일/디렉토리를 이미지 안에 복사하는 기능 (단순복사 기능) ex. COPY index.html /usr/share/nginx/html
ADD	복사 기능 및 URL이용한 이미지 설정, 자동 압축 해제 기능 ex. ADD index.html /usr/share/nginx/html ADD http://example.com/cust.tar.gz /work/data ADD web.tar.gz /var/www/html

6.2 주요 명령어

명령어	설명
ENV	이미지 안에 환경 변수 설정. RUN, WORKDIR 에서 참조가능 ex. ENV JAVA_HOME /usr/lib/jvm/java-21
EXPOSE	컨테이너가 호스트 네트워크를 통해 들어오는 트래픽 리스닝하는 포트와 프로토콜 번호를 지정된 번호로 노출할 것임을 그냥 문서화하는 기능임. 하지만 docker로 실행할 때는 반드시 -p 로 포트번호를 실제로 노출 필요. ex. EXPOSE 80
VOLUME	익명 볼륨 을 이미지 빌드에 미리 설정하는 경우 작성 (컨테이너 삭제 시 제거됨) ex. VOLUME /var/www/html (-v /var/www/html 와 동일)
WORKDIR	컨테이너에서 작업할 기준 경로(디렉터리) 설정. RUN, CMD, COPY, ADD 명령문은 해당 디렉터리를 기준으로 실행됨 ex. WORKDIR /workspace
CMD	컨테이너가 실행될 때 기본으로 실행할 명령 또는 인자를 정의 docker run 시 명령어를 쉽게 덮어 사용 가능 ex. CMD ["python3", "app.py"] docker run my-python python3 hello.py 하면 CMD는 대체됨
ENTRYPOINT	컨테이너가 실행될 때 반드시 실행되는 고정된 명령 정의 ex. ENTRYPOINT ["npm" , "start"] ENTRYPOINT ["python"] CMD ["app.py"]

6.3 실행 명령어

RUN vs CMD , ENTRYPOINT

- 공통적으로 모두 실행하는 명령어

구분	실행 시점	실행 명령어	역할
RUN	image 빌드 시	docker build	image에 명령어 실행
CMD	Container 실행 시	docker run	기본 실행 명령 (교체가능)
ENTRYPOINT	Container 실행 시	docker run	고정 실행 명령

6.4 이미지 빌드

이미지 빌드 명령어

- 개념: Dockerfile 파일을 이용하여 이미지를 빌드하는 명령어.
- 문법:

docker buildx build [OPTIONS] PATH | URL | -

별칭: docker build, docker builder build, docker image build

명령어	설명
옵션	-t 옵션: " 이미지명:태그 " 를 지정하는 경우에 사용 -f 옵션: Dockerfile 명이 아닌 임의의 파일명을 사용하는 경우 ex. docker run -t myapp:1.0 -f Dockerfile-dev
경로(path)	디렉터리 단위 개발을 권고. 현재 경로에 Dockerfile이 있으면 . (현재 디렉터리)사용
URL	Dockerfile이 포함된 github URL을 제공 ex. docker build -t myapp:1.0 github.com/user/docker-web

6.4 이미지 빌드

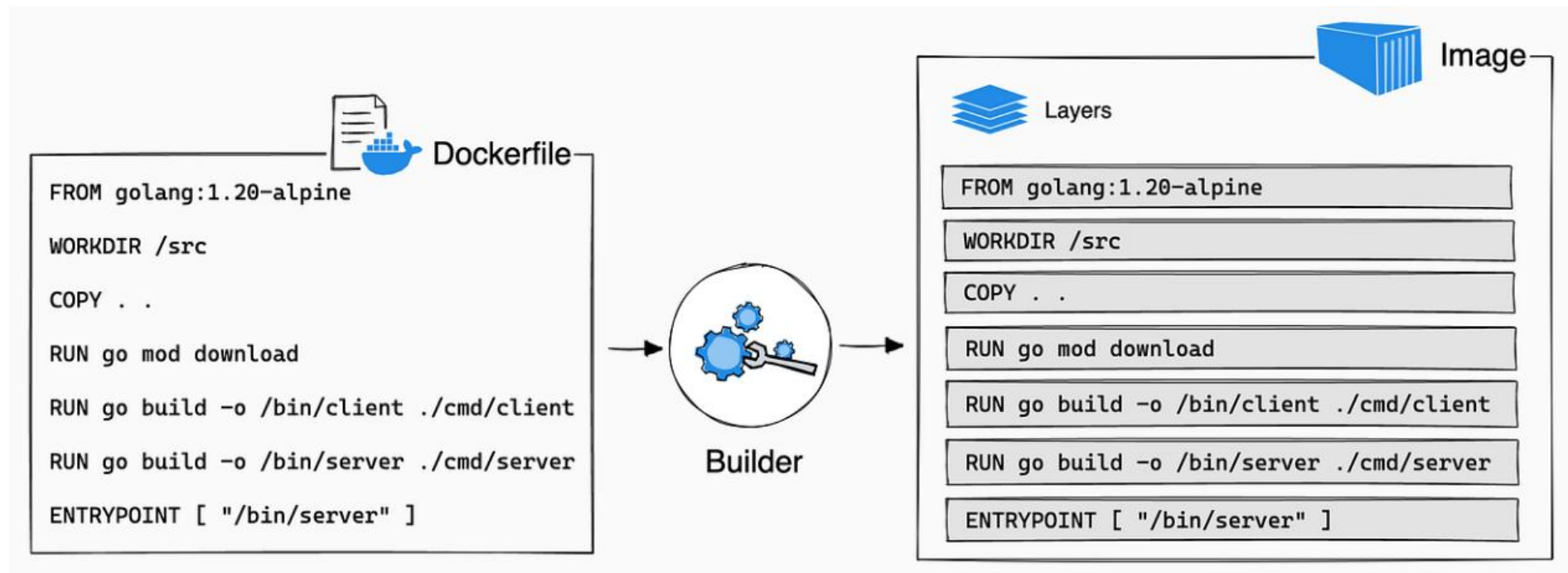
이미지 빌드 내부 동작

- **docker build -t my-app .**
- 현재 디렉터리 구성 파일은 Dockerfile, app.py, config/ , requirements.txt

동작 순서	동작	설명
1	Build Context 전송	Docker build 시 Docker 데몬에게 전달되는 파일들 범위. 현재 디렉터리 구성 파일들 모두 Docker 데몬으로 전달됨. 따라서 용량 큰 파일 포함 시 build가 느려짐 .env, .git, node_modules 제외 (.dockerignore 필수)
2	Dockerfile 읽기	FROM ubuntu:22.04 WORKDIR /app COPY . . RUN apt update && apt install -y python3 CMD ["python3", "app.py"]
3	명령어 한 줄씩 실행	FROM ubuntu:22.04 WORKDIR /app
4	각 단계마다 Image Layer 생성	FROM ubuntu:22.04 <-- Image Layer 1 WORKDIR /app <-- Image Layer 2
5	최종 Image 생성	docker images

6.4 이미지 빌드

이미지 빌드 내부 동작



6.4 이미지 빌드

실습1

- 제공된 dockerfile_exam.zip 이용
- 명령어: `docker build -t my-app .`
`docker run -it -d -p 8080:80 my-app`

FROM node:14-alpine

WORKDIR /app

COPY package.json .

RUN npm install

COPY . .

EXPOSE 80

CMD ["node", "app.js"]

```
C:\docker_study\dockerfile_exam>docker build -t my-app .
[+] Building 13.2s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 153B
=> [internal] load metadata for docker.io/library/node:14-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 69B
=> [1/5] FROM docker.io/library/node:14-alpine@sha256:434215b487a329c9e867202ff8
=> => resolve docker.io/library/node:14-alpine@sha256:434215b487a329c9e867202ff8
=> => sha256:561cb69653d56a9725be56e02128e4e96fb434a8b4b4decf2bdeb479a225feaf 44
=> => sha256:e5fca6c395a62ec277102af9e5283f6edb43b3e4f20f798e3ce7e425be226ba6 2.
=> => sha256:8f665685b215c7daf9164545f1bbdd74d800af77d0d267db31fe0345c0c8fb8 37.
=> => sha256:f56be85fc22e46face30e2c3de3f7fe7c15f8fd7c4e5add29d7f64887abdaa09 3.
=> => extracting sha256:f56be85fc22e46face30e2c3de3f7fe7c15f8fd7c4e5add29d7f64b8
=> => extracting sha256:8f665685b215c7daf9164545f1bbdd74d800af77d0d267db31fe0345
=> => extracting sha256:e5fca6c395a62ec277102af9e5283f6edb43b3e4f20f798e3ce7e425
=> => extracting sha256:561cb69653d56a9725be56e02128e4e96fb434a8b4b4decf2bdeb479
=> [internal] load build context
=> => transferring context: 31.73kB
=> [2/5] WORKDIR /app
=> [3/5] COPY package.json .
=> [4/5] RUN npm install
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:92405f34341e121f346fc4967643eb8cd0571c0fcb1eaeffc
=> => exporting config sha256:04286d858aeb0f319616e0b23ad0c58444e52dd074e4a60b32
=> => exporting attestation manifest sha256:277dacfd4c11eb3910ab5e16a91759d9b2b4
=> => exporting manifest list sha256:d7ba9336b2d1a9c05bab01e545a297342bef3c9a3cc
=> => naming to docker.io/library/my-app:latest
=> => unpacking to docker.io/library/my-app:latest
```

6.4 이미지 빌드

실습1

- 제공된 welcome.html 파일 수정 후 다시 build 한다.
- 명령어: **docker build -t my-app .**
docker run -it -d -p 8080:80 my-app

```
C:\docker_study\dockerfile_exam>docker build -t my-app .
[+] Building 2.0s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 153B
=> [internal] load metadata for docker.io/library/node:14-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 69B
=> [1/5] FROM docker.io/library/node:14-alpine@sha256:434215b487a321
=> => resolve docker.io/library/node:14-alpine@sha256:434215b487a321
=> [internal] load build context
=> => transferring context: 637B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY package.json .
=> CACHED [4/5] RUN npm install
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:7214f79599fc49ef8b1b0b2d5b88ead2c54
=> => exporting config sha256:6efbba4b40c2e9d9281cf1b298768b7e7f684
=> => exporting attestation manifest sha256:cc5bda88eedda5519ad97a5
=> => exporting manifest list sha256:b77e0445091c098ff20c0da36cdb58
=> => naming to docker.io/library/my-app:latest
=> => unpacking to docker.io/library/my-app:latest
```

**캐시되어
매우 빠르게 빌드됨**

6.4 이미지 빌드

실습2

- 제공된 boot_Dockerfile.zip 이용
- 명령어: **gradlew clean build**

docker build -t springboot21 .

docker run -d --name spring21 -p 8080:8080 springboot21

FROM eclipse-temurin:21-jre

WORKDIR /app

COPY build/libs/*.jar app.jar

EXPOSE 8080

**ENTRYPOINT ["java", "-jar",
"app.jar"]**

```
C:\springboot3_vuejs_study\boot_Dockerfile>docker build -t sprintboot21 .
[+] Building 4.1s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 172B
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 306B
=> [1/3] FROM docker.io/library/eclipse-temurin:21-jre@sha256:ff65ff0d43c73d2b6
=> => resolve docker.io/library/eclipse-temurin:21-jre@sha256:ff65ff0d43c73d2b6
=> [internal] load build context
=> => transferring context: 21.00MB
=> [2/3] WORKDIR /app
=> [3/3] COPY build/libs/*.jar app.jar
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:a1de2e716c0cc08ce348873e55f30bcd3ce97cb6aab2c2d
=> => exporting config sha256:ccf786d0d6d0df273e80c0a3f3848e11fd6a4a09b3076ab14
=> => exporting attestation manifest sha256:debe28631f0b683b44086d291a9faadd7b7
=> => exporting manifest list sha256:e0cd069c8b9896d2a2cc5fcc2dc0c6348da061a089
=> => naming to docker.io/library/sprintboot21:latest
=> => unpacking to docker.io/library/sprintboot21:latest
```

6.5 Multi-stage Dockerfile 패턴

개념

- 빌드에 필요한 것과 실행에 필요한 것을 단계(stage)로 분리해서, 이미지를 작게/안전하게/재현 가능하게 만드는 Dockerfile작성 패턴이다.

구현

- 1단계(Builder)
 - 컴파일/빌드 도구(Gradle, Maven, Node 등)까지 포함해서 산출물(jar, dist 등) 만듦.
- 2단계(Runner)
 - 실행에 필요한 최소 런타임(JRE, nginx 등)만 두고 1단계 결과물만 복사해서 실행.

장점

- 이미지 크기 감소
- 보안 개선 (빌드 툴/소스 코드가 런타임 이미지에 없음)
- 빌드 환경이 런타임 환경에 섞이지 않아서 가독성 증가
- 캐시 활용 최적화 가능

6.5 Multi-stage Dockerfile 패턴

샘플

```
FROM node:14-alpine
```

```
WORKDIR /app
```

```
COPY package.json .
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 80
```

```
CMD ["node", "app.js"]
```



```
# 1) deps stage: 의존성 설치 전용
```

```
FROM node:14-alpine AS deps
```

```
WORKDIR /app
```

```
COPY package.json ./
```

```
RUN npm install --only=production
```

```
# 2) runner stage: 실행 전용 (최종 이미지)
```

```
FROM node:14-alpine AS runner
```

```
WORKDIR /app
```

```
# deps 스테이지에서 설치된 node_modules만 가져오기
```

```
COPY --from=deps /app/node_modules ./node_modules
```

```
# 소스 복사
```

```
COPY . .
```

```
EXPOSE 80
```

```
CMD ["node", "app.js"]
```

7.1 Docker Network 개요

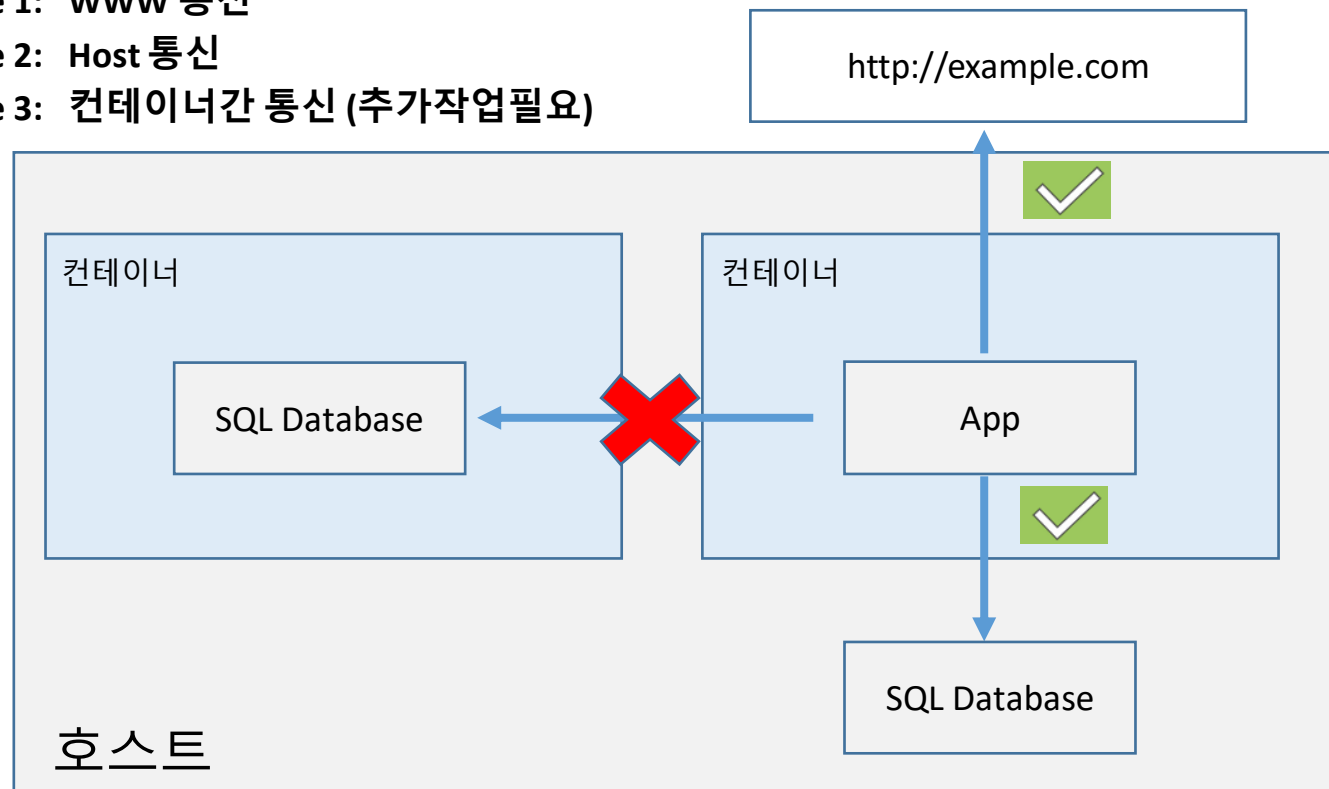
Docker Network

- 개념

- 컨테이너들간 서로 통신하고, 외부 통신도 할 수 있도록 만들어주는 가상 네트워크 환경.

- 통신 형태

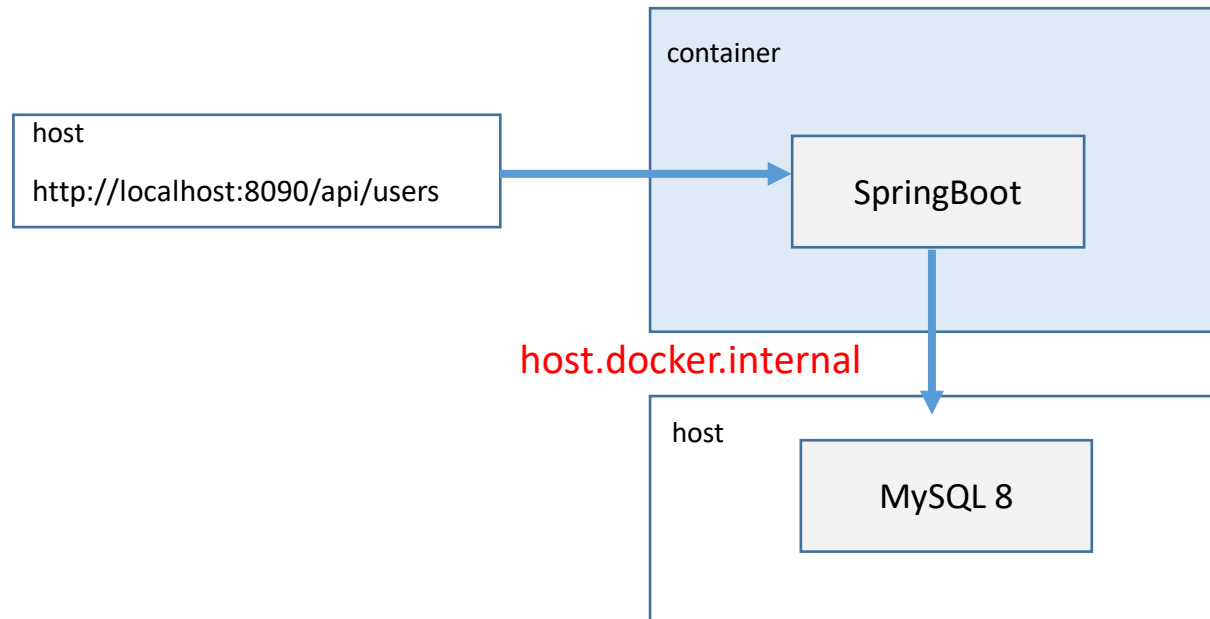
- case 1: WWW 통신
- case 2: Host 통신
- case 3: 컨테이너간 통신 (추가작업필요)



7.1 Docker Network 개요

실습1

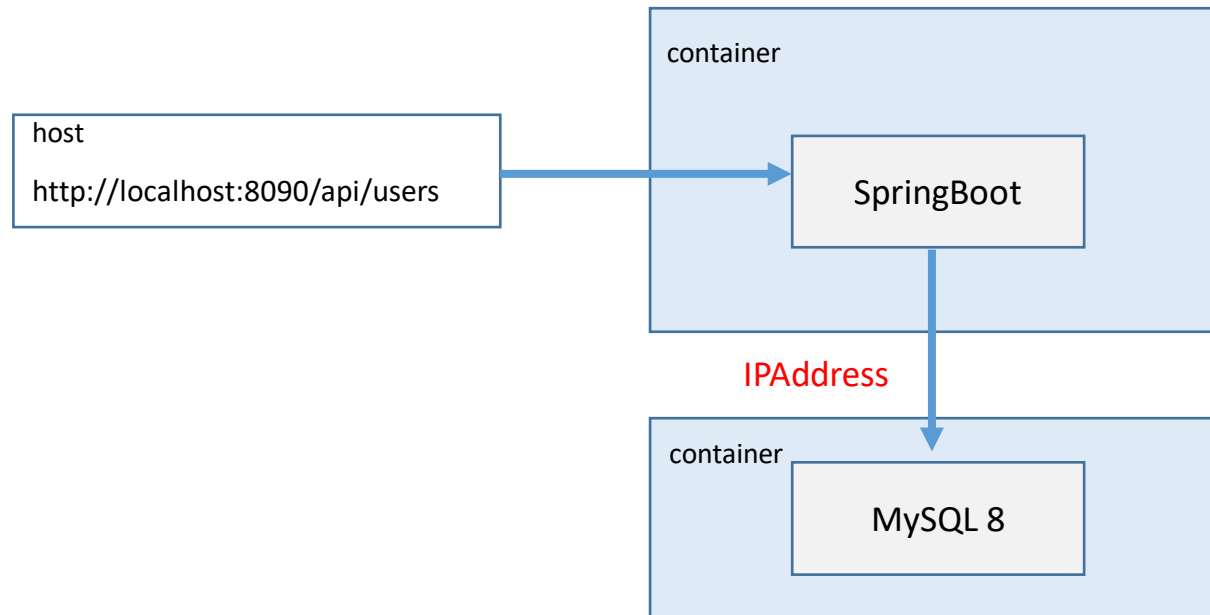
- 제공된 boot_Dockerfile4_Network_mysql1_host.zip 이용
- 명령어: **gradlew clean bootJar -x test**
docker build -t spring-mysql-demo .
docker run -d --name spring-mysql-app -p 8090:8090 spring-mysql-demo



7.1 Docker Network 개요

실습2 컨테이너간 IPAddress로 접근

- 제공된 boot_Dockerfile4_Network_mysql2_container.zip 이용
- 명령어: **gradlew clean bootJar -x test**
docker build -t spring-mysql-demo2 .
docker run -d --name spring-mysql-app2 -p 8090:8090 spring-mysql-demo2



7.1 Docker Network 개요

실습2 MySQL 데이터베이스 (볼륨 + env)

```
docker volume create mysql-data-vol2
```

```
docker volume ls
```

```
docker run -it -d --name mysql-data2 -e MYSQL_ROOT_PASSWORD=1234 -e MYSQL_DATABASE=testdb  
-v mysql-data-vol2:/var/lib/mysql mysql:8.3
```

docker container inspect mysql-data2 # IPAddress 조회해서 SpringBoot에서 사용

```
docker exec -it mysql-data2 bash
```

```
bash-4.2# mysql -u root -p
```

```
Enter password:
```

```
mysql> show databases;
```

```
mysql> use testdb;
```

```
Database changed
```

7.2 Docker Network 생성

여러 컨테이너들을 동일 Network로 묶기

- 개념

- 네트워크로 묶인 모든 컨테이너들은 IP가 자동으로 인식되어 서로 간에 통신이 가능해짐.

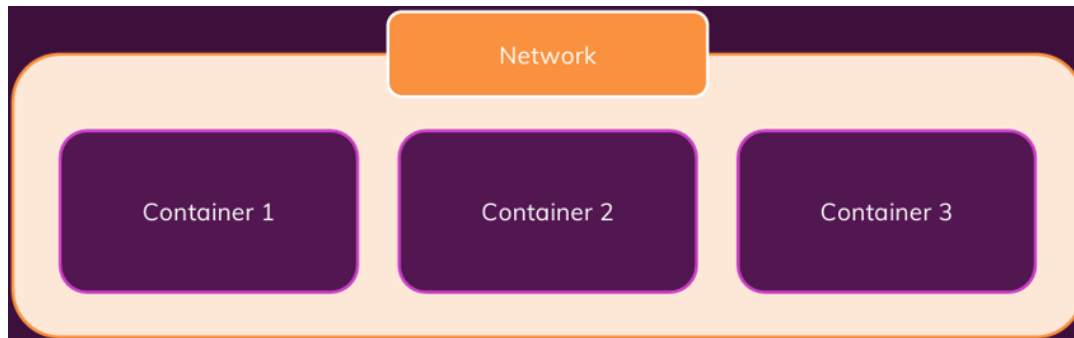
- 특징

- 동일 네트워크로 묶인 컨테이너들은 IPAddress 대신에 타겟 컨테이너명을 도메인으로 활용 가능.
즉 IPAddress 대신에 컨테이너명을 사용하여 접속이 가능해짐.

ex. `dbc:mysql://172.17.0.2:3306/testdb` 대신에 `dbc:mysql://mysql:3306/testdb` 사용

- 문법

- `docker network create` 네트워크명
`docker run --network` 네트워크명



7.2 Docker Network 생성

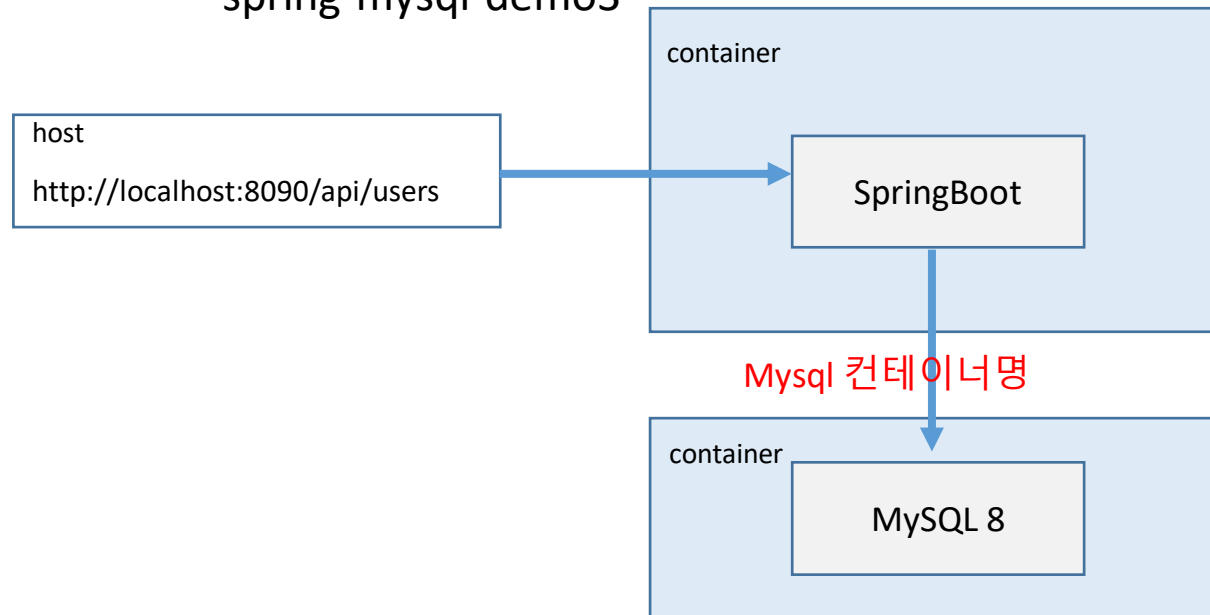
실습3 컨테이너간 타겟 이름으로 접근

- 제공된 boot_Dockerfile4_Network_mysql3_network.zip 이용
- 명령어: `gradlew clean bootJar -x test`

`docker network create boot-net`

`docker build -t spring-mysql-demo3 .`

`docker run -d --name spring-mysql-app3 -p 8090:8090 --network boot-net
spring-mysql-demo3`



7.2 Docker Network 생성

실습3 MySQL 데이터베이스 (볼륨 + env)

```
docker volume create mysql-data-vol3
```

```
docker volume ls
```

```
docker run -it -d --name mysql-data3 -e MYSQL_ROOT_PASSWORD=1234 -e  
MYSQL_DATABASE=testdb -v mysql-data-vol3:/var/lib/mysql --network boot-net mysql:8.3
```

```
docker exec -it mysql-data3 bash
```

```
bash-4.2# mysql -u root -p
```

```
Enter password:
```

```
mysql> show databases;
```

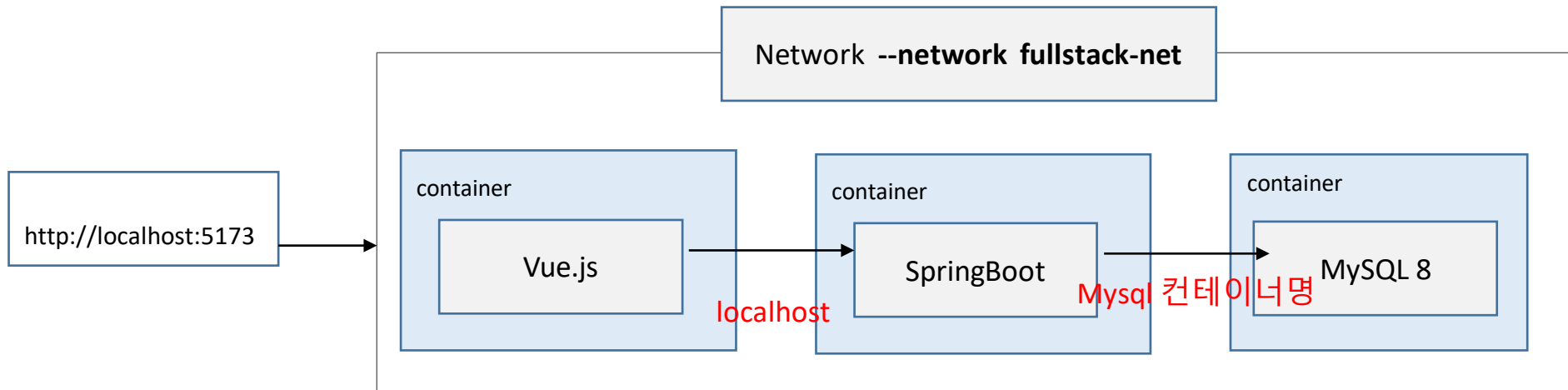
```
mysql> use testdb;
```

```
Database changed
```

7.2 Docker Network 생성

실습4 FullStack

- 제공된 boot_backend_Dockerfile.zip 이용



7.2 Docker Network 생성

실습4 FullStack

- 제공된 boot_backend_Dockerfile.zip 이용
- 명령어:

```
docker network create fullstack-net
```

```
docker run -d --name redis-server -p 6379:6379 --network fullstack-net redis
```

```
docker build -t springboot-backend .
```

```
docker run -d --name springboot-backend-app -p 8090:8090 --network fullstack-net  
springboot-backend
```

7.2 Docker Network 생성

실습4 MySQL 데이터베이스 (볼륨 + env)

```
docker volume create mysql-data-vol4
```

```
docker volume ls
```

```
docker run -it -d --name mysql-data4 -e MYSQL_ROOT_PASSWORD=1234 -e  
MYSQL_DATABASE=testdb -v mysql-data-vol4:/var/lib/mysql --network fullstack-net mysql:8.3
```

```
docker exec -it mysql-data4 bash
```

```
bash-4.2# mysql -u root -p
```

```
Enter password:
```

```
mysql> show databases;
```

```
mysql> use testdb;
```

```
Database changed
```

7.2 Docker Network 생성

실습4 Vuejs

제공된 boot_frontend_Dockerfile.zip 이용

```
docker build -t vue-frontend .
```

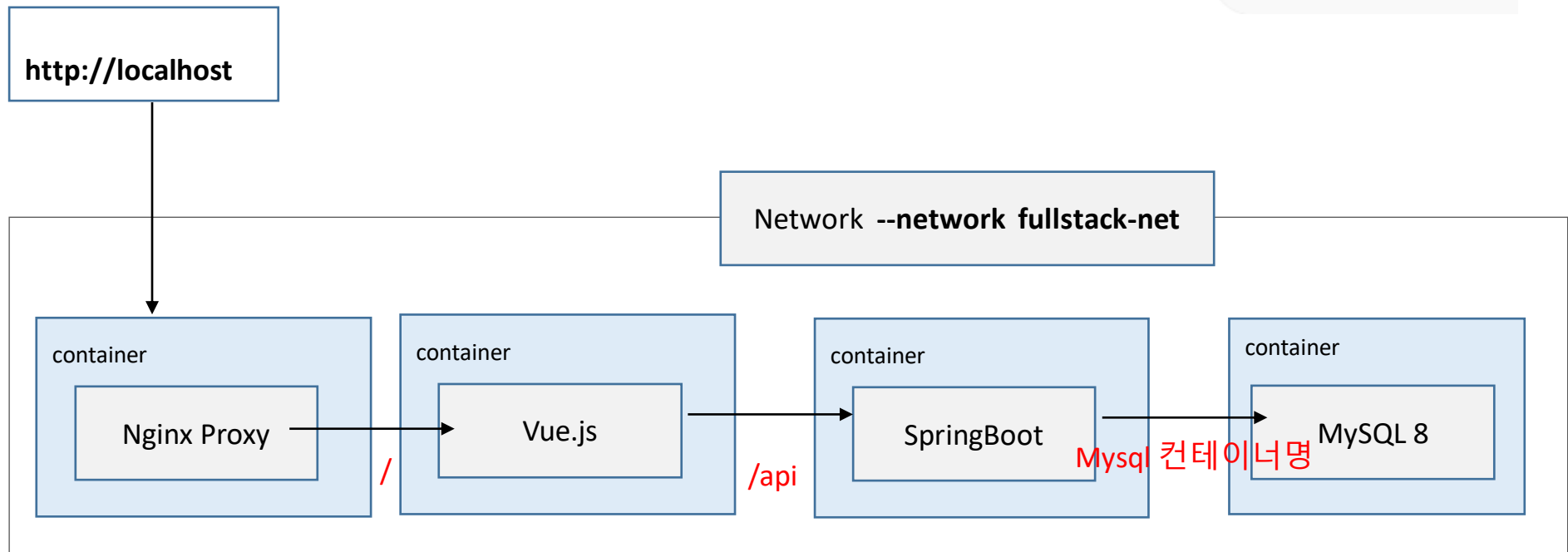
```
docker run -p 5173:5173 --network fullstack-net vue-frontend
```

7.3 Docker Network 생성 + Nginx Proxy

실습5 FullStack + Nginx Proxy

- 제공된 boot_backend_Dockerfile2_nginx_proxy.zip 이용
boot_frontend_Dockerfile2_nginx_proxy.zip
nginx.zip

```
fullstack/  
├─ backend/  
├─ frontend/  
└─ nginx/  
    └─ nginx.conf
```



7.3 Docker Network 생성 + Nginx Proxy

실습5 FullStack + Nginx Proxy

```
docker volume create mysql-data-vol5
```

```
docker volume ls
```

```
docker run -it -d --name mysql-data5 -e MYSQL_ROOT_PASSWORD=1234 -e  
MYSQL_DATABASE=testdb -v mysql-data-vol5:/var/lib/mysql --network fullstack-net  
mysql:8.3
```

```
docker run -d --name redis-server -p 6379:6379 --network fullstack-net redis
```

```
docker build -t springboot-backend .
```

```
docker run -d --name springboot-backend-app -p 8090:8090 --network fullstack-net  
springboot-backend
```

```
docker build -t vue-frontend .
```

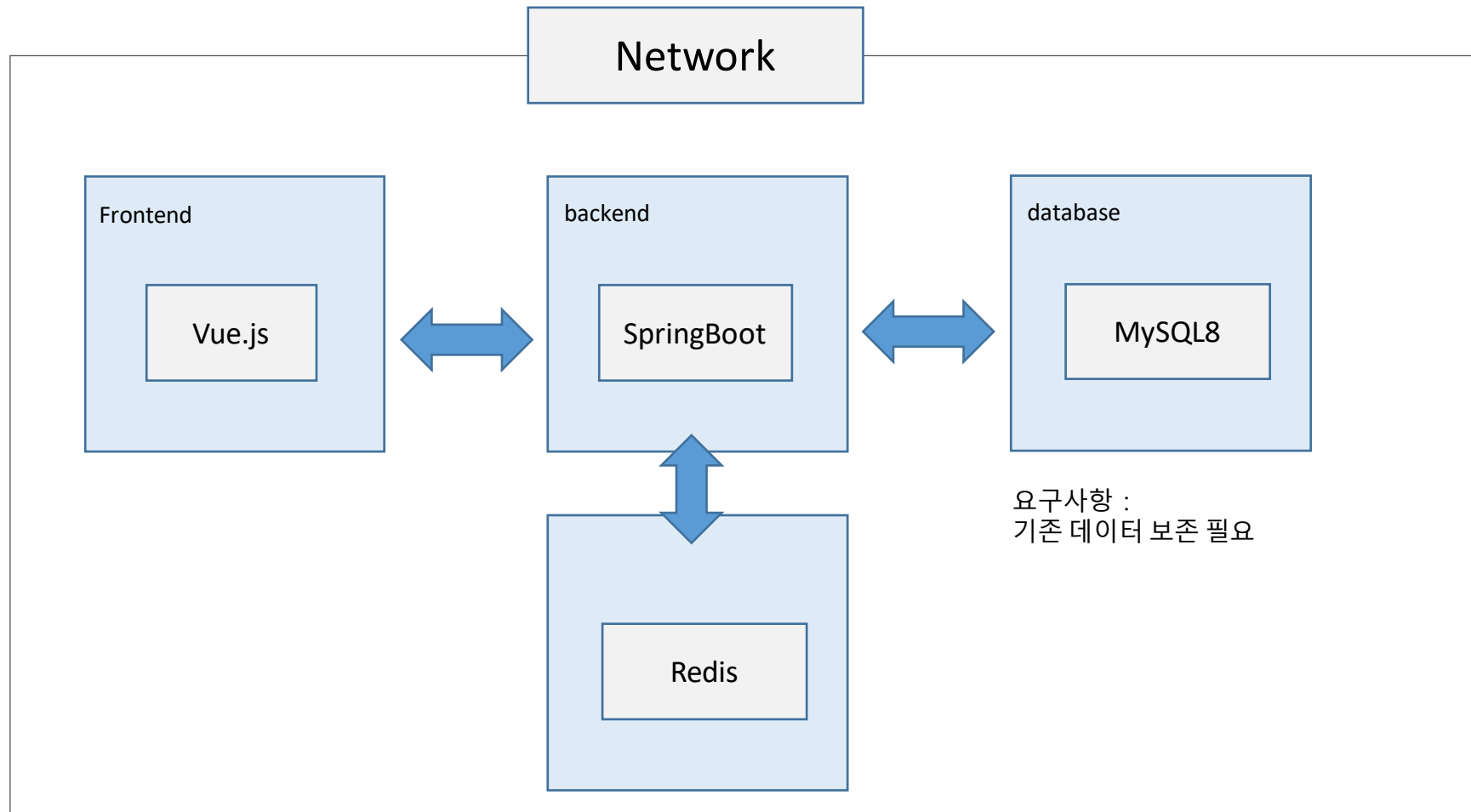
```
docker run -d --name vue-frontend -p 5173:5173 --network fullstack-net vue-frontend
```

#powershell

```
docker run -d --name nginx-proxy -p 80:80 --network fullstack-net -v  
"${PWD}/nginx/nginx.conf:/etc/nginx/conf.d/default.conf" ` nginx:1.27-alpine
```


8.1 Docker Compose

Multi Application Architecture



8.1 Docker Compose

Docker Compose

- 개념

- 여러 개의 컨테이너(서비스)를 한 번에 정의하고, 한 번에 실행/중지/삭제가 가능하도록 지원하는 도구

- 파일명

- docker-compose.yaml
- docker-compose.yml
- compose.yaml
- compose.yml

- 특징

- 앞서 사용했던 Dockerfile 파일을 대체하지 않고 함께 사용됨
- Docker Image 또는 Docker Container를 대체하지 않고 기존보다 훨씬 쉽게 사용 가능
- 하나의 호스트에서 다수의 컨테이너를 관리하는데 매우 우수
- --network 옵션을 설정하지 않아도 자동으로 동일 네트워크로 묶임

8.1 Docker Compose

Docker Compose 파일 기본 구조 예

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    depends_on:
      - db

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: 1234
      MYSQL_DATABASE: testdb
    volumes:
      - dbdata:/var/lib/mysql

volumes:
  dbdata:
```

8.2 service 속성

service 속성

• 개념

- 컨테이너 대신에 service라는 최상위 개념을 사용

항목	설명
services	docker compose에서 컨테이너(서비스) 묶음의 정의 영역
의미	하나의 서비스는 일반적으로 하나의 컨테이너임
핵심 포인트	서비스 이름 = 컨테이너 이름 (내부 DNS 호스트명)

• 주요 하위 속성

```
services:
  service 명
  image
  build
  container_name
  command
  entrypoint
  ports
  expose
  networks
  environment
```

```
environment
  env_file
  volumes
  depends_on
  working_dir
  ...
```

8.3 service 주요 하위 속성

이미지 / 빌드 관련 속성

• image 속성

항목	설명
역할	사용할 Docker 이미지 지정
형식	이미지명:태그
이미지 저장소	Docker Hub / Private Registry
사용 예	image: nginx:alpine

• build 속성

항목	설명
역할	Dockerfile 이 있는 경우에 사용하며 이미지를 직접 빌드함
특징	어플리케이션을 직접 개발할 때 사용됨.
하위 속성	context 속성: Dockerfile 기준 경로 dockerfile 속성: Dockerfile 이름
사용 예	build: context: . dockerfile: Dockerfile

8.3 service 주요 하위 속성

컨테이너 실행 제어 속성

- **container_name 속성**

항목	설명
역할	컨테이너 이름 직접 지정
특징	실무에서는 권장 안함
사용 예	<code>container_name: my-nginx</code>

- **command 속성**

항목	설명
역할	이미지의 기본 CMD 덮어씀
사용 용도	서버 실행 방식 변경
사용 예	<code>command: ["node", "server.js"]</code>

- **entrypoint 속성**

항목	설명
역할	ENTRYPOINT 자체를 변경
사용 예	<code>entrypoint: ["npm", "run", "dev"]</code>

8.3 service 주요 하위 속성

네트워크 / 포트 속성

• ports 속성

항목	설명
역할	호스트와 컨테이너 간 포트 연결 (port forwarding)
형식	호스트:컨테이너
사용 예	ports: - "80:80"

• expose 속성

항목	설명
역할	내부 네트워크에만 포트 공개
특징	외부 접근 불가, 문서화 + 내부 통신
사용 예	expose: - "3306"

• networks 속성

항목	설명
역할	컨테이너가 연결될 네트워크. (자동 생성됨)
사용 예	networks: - backend

8.3 service 주요 하위 속성

환경 변수 속성

- **environment 속성**

항목	설명
역할	컨테이너 내부 환경 변수
사용 용도	DB 정보
사용 예	environment: - MYSQL_DATABASE: appdb

- **env_file 속성**

항목	설명
역할	외부 환경변수 파일 불러오기
특징	민감한 정보 분리. .env 권장
사용 예	env_file: - .env

8.3 service 주요 하위 속성

데이터 유지 속성

- **volumes 속성**

항목	설명
역할	데이터 영속성 보장
사용 용도	DB, 로그 등
사용 예	volumes: - dbdata:/var/lib/mysql

8.3 service 주요 하위 속성

서비스 의존성 / 실행 순서 속성

- **depends 속성**

항목	설명
역할	실행 순서 제어
특징	시작 순서 보장
사용 예	depends_on: - db

8.3 service 주요 하위 속성

재시작 / 안정성 속성

• restart 속성

항목	설명
옵션	always: 항상 재시작 unless-stopped : 수동 중지 시 재시작 on-failure : 실패 시만 재시작 no : 재시작 안함
사용 예	restart: always

• healthcheck 속성

항목	설명
역할	살아있음이 아닌 정상 동작 중(healthy) 체크에 주안점
하위 속성	test: 상태 검사 명령 interval: 검사 주기 (기본값 30s) timeout: 검사 타임아웃 (기본값 30s) retries: 실패 허용 횟수 (기본값 3) start_period: 초기 유예 시간 (기본값 0s)
사용 예	healthcheck: test: ["CMD", "mysqladmin", "ping", "-h", "localhost"] interval: 5s timeout: 3s retries: 10 start_period: 30s

8.3 service 주요 하위 속성

개발 / 터미널 속성

- **working_dir 속성**

항목	설명
역할	컨테이너 작업 디렉터리
사용 예	working_dir: /app command: node server.js # /app/server.js 실행

- **stdin_opn / tty 속성**

항목	설명
역할	stdin_opn : 표준 입력 유지 tty: 터미널 모드
사용 예	stdin_open: true tty: true

8.4 Docker Compose 주요 실행 명령어

실행 명령어

명령어	설명	특징
docker compose up docker compose up -d docker compose up 서비스명	서비스 실행 (foreground) 서비스 실행 (background) 특정 서비스만 실행(선택 실행)	
docker compose down docker compose down -v	컨테이너 중지 + 삭제 완전 초기화	볼륨은 유지됨 볼륨 삭제됨
docker compose stop docker compose stop 서비스명 docker compose start docker compose restart	서비스 일시 중지 특정 서비스만 중지(선택 중지) 다시 시작 전체 재시작	컨테이너 중지 특정 컨테이너만 중지 정지 컨테이너 재시작 stop + start
docker compose ps	상태 확인 (running, healthy, exited)	healthcheck 확인
docker compose logs docker compose logs 서비스명	모든 서비스 로그 특정 서비스만 로그	디버깅 서비스 로그 분리
docker compose build docker compose up --build	이미지 빌드 빌드 후 실행	Dockerfile 실행 build + up
docker compose exec	실행 중 컨테이너 접속	
docker compose config docker compose config --services	설정 정보 병합 결과 서비스 목록만	디버깅 구조 파악
docker compose rm	중지된 컨테이너만 삭제	

8.5 Docker Compose 실습

실습

```
▼ project
  > backend
  > frontend
  ▼ nginx
    📄 Dockerfile
    ⚙️ nginx.conf
    📄 docker-compose.yml
```

```
services:
  mysql-data5:
    image: mysql:8.3
    container_name: mysql-data5
    environment:
      MYSQL_ROOT_PASSWORD: 1234
      MYSQL_DATABASE: testdb
    volumes:
      - mysql-data-vol5:/var/lib/mysql
    networks:
      - fullstack-net

  redis-server:
    image: redis
    container_name: redis-server
    ports:
      - "6379:6379"
    networks:
      - fullstack-net
```

8.5 Docker Compose 실습

실습

```
springboot-backend-app:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: springboot-backend-
app
  ports:
    - "8090:8090"
  depends_on:
    - mysql-data5
    - redis-server
  networks:
    - fullstack-net

vue-frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: vue-frontend
  ports:
    - "5173:5173"
  depends_on:
    - springboot-backend-app
  networks:
    - fullstack-net
```

```
nginx-proxy:
  build:
    context: ./nginx
    dockerfile: Dockerfile
  container_name: nginx-proxy
  ports:
    - "80:80"
  depends_on:
    - vue-frontend
    - springboot-backend-app
  networks:
    - fullstack-net

volumes:
  mysql-data-vol5:

networks:
  fullstack-net:
    driver: bridge
```

수고하셨습니다.